



루프를 닫는다

답에서 증명서로, 수학 추론과 Lean 검증 사이 — Magenta(arXiv 2609.11319) 정독

- 비형식 추론과 Lean 검증 사이 루프 — 명제 판사와 오류 귀인 판사가 닫는다
- AIME 2025·2026·HMMT 2026 전 벤치마크 100% — 오픈 웨이트 7B로 IMO 2026 전 문제
- 원문 그림 6종 9면 그대로 수록 + 표 7종 한국어 재구성, 오류 해부와 절제 분석 정독

차례

들어가며	3
01 검증 없는 답은 반쪽이다	6
02 일곱 구성요소로 본 파이프라인	9
03 루프는 이렇게 돈다	12
04 판사 둘과 보증의 경계	14
05 실험 설계 — 모델, 벤치마크, 예산	16
06 표로 보는 완승	19
07 7B가 IMO를 넘었다	22
08 명제 생성의 경제학	25
09 비용 곡선의 네 패널	28
10 바뀌 말해도 무너지지 않는다	33
11 판사를 끼웠을 때와 꺾을 때	35
12 고치기와 다시 뽑기	38
13 오류의 해부 — 실패 지형과 형식화 갭	40
14 한국 현장에서 읽는 Magenta	43
15 맺음말	45

DAY

들어가며



“원문: Magenta: Closing the Loop Between Mathematical Reasoning and Lean Verification — arXiv 2609.11319v1(2026) · Joshua Ong Jun Leang·Haonan Li·Zheng Zhao·Xinyi Shang·Wenda Li·Zhengzhong Liu·Eric Xing·Shay B. Cohen·Eleonora Giunchiglia(MBZUAI Institute of Foundation Models·University of Edinburgh·University College London·Imperial College London)”

“링크: <https://arxiv.org/abs/2609.11319>”

■ 답에서 증명서로

대형 언어 모델은 수학 문제를 잘 푼다. 그러나 잘 푼다는 말과 옳다는 말은 다르다. 모델이 내놓는 추론 사슬은 길고 그럴듯하지만, 그 안에는 오류와 막다른 길이 상시 섞여 있다. 사람이 판독해야만 겨우 걸러지는 이 불확실성이, 수학을 실무의 도구로 쓰는 일을 늘 가로막아 왔다. 이 논문이 출발하는 지점이 바로 여기다. 기계는 검산을 할 수 있다. 증명 보조기 Lean 4가 명제를 받아들이는 순간, 그 증명은 사람의 개입 없이 옳음이 보장된다. 그런데 지금까지의 언어 모델은 이 검증 능력을 거의 쓰지 못한 채 비형식 추론에만 매달려 왔다.

논문이 내놓는 Magenta는 이 빈자리를 채우는 파이프라인이다. 자연어로 쓰인 문제만 넣으면, 추론기가 답을 만들고, 형식화기가 그 문제와 답을 Lean 4 명제로 옮기고, 증명기가 기계가 검사하는 증명을 완성한다. 학습도 튜닝도 없이 프롬프트로만 일곱 구성요소를 조립했다는 점이 이채롭다. 관건은 두 명의 판사다. 명제 판사(statement judge)는 “이 Lean 명제가 원래 문제와 답을 충실하게 보존하는가”를 심사해, 증명은 됐는데 다른 것을 증명한 거짓 증명서(false certificate)를 차단한다. 오류 귀인 판사(error attribution judge)는 검증이 실패했을 때 “잘못이 수학에 있는가, Lean 구현에 있는가”를 가려 갈 길을 정한다. 수학이 틀렸으면 처음부터 다시 추론하고, 구현만 틀렸으면 증명만 고친다.

결과는 단호하다. AIME 2025, AIME 2026, HMMT 2026 세 벤치마크 93문제 전부를 네 종류의 추론기에서 100퍼센트로 검증했고, 오픈 웨이트 7B 모델 K2-Horizon-7B와 결합해 IMO 2026 여섯 문제 전부를 풀었다. 논문의 절제 실험은 두 가지 교훈도 남긴다. 명제 심사를 끄면 반환된 증명서의 절반 가까이가 거짓이 되고, 컴파일 피드백을 버리고 다시 뽑기만 하면 어려운 문제에서 무너진다. 검증 피드백을 조건으로 삼는 자기수정이 독립 재시도를 이끈다는 것이다.

■ 이 책의 구성

1장부터 4장까지는 방법의 정독이다. 검증 없는 답이 왜 반쪽인지에서 출발해(1장, 그림 1), 일곱 구성요소가 각각 무슨 일을 하는지(2장), 세 겹의 루프가 어떻게 도는지(3장), 두 판사가 보증의 경계를 어디까지 미는지(4장)를 차례로 연다.

5장부터 7장까지는 실험이다. 모델·벤치마크·예산의 설계(5장)를 지나 완승의 표(6장), 그리고 7B 모델이 IMO를 넘는 장면(7장, 그림 2·6)까지 결과 전체를 훑는다.

8장부터 13장까지는 분석과 오류의 해부다. 명제 생성의 경제학(8장, 그림 3·4), 비용 곡선의 네 패널(9장, 그림 5), 바뀌 말해도 무너지지 않는 강건성(10장), 판사를 끄고 켜는 절제 실험(11장), 고치기와 다시 뽑기의 대결(12장), 그리고 실패의 지형을 표 세 장으로 해부한다(13장).

14장은 한국 현장의 각색이고, 15장이 맺음말이다. 이 책의 그림은 원문 그림 여섯 종을 그대로 실었고, 표는 원문 수치를 그대로 담아 한국어로 다시 지었다. 13장의 두 표는 원문 표의 행을 압축해 주요 범주만 남겼고, 남긴 셀의 수치는 원문과 같다.

■ 각색에 대하여

분명히 둔다. 14장의 비유와 적용은 원문 저자의 주장이 아니다. 원문은 수학 추론과 Lean 검증을 다룰 뿐, 한국의 개발 문화나 실무 파이프라인을 이야기하지 않는다. 그 장은 이 책을 엮으면서 필자가 원문의 개념 — 증명서, 판사의 이중화, 재시

도 예산의 비대칭 — 을 우리 현장에 옮겨 본 것이다. 원문의 수치와 사실은 그대로 옮겼고, 필자의 해석이 개입하는 지점에서는 본문에서 그렇게 밝힌다.

I 읽는 법

바쁜 독자는 세 장만 읽어도 논문의 뼈대가 잡힌다. 파이프라인 전체가 그려진 1장, 완승의 표가 있는 6장, 그리고 판사를 꺾을 때 무슨 일이 벌어지는지 보여 주는 11장이다. 나머지 장은 이 세 장이 남긴 질문에 하나씩 답한다. 그림으로 읽고 싶은 독자는 각 장에 실린 원문 그림 여섯 종을 먼저 훑어도 좋다. 그림 1이 전체 지도고, 나머지 그림이 그 지도 위에서 루프가 실제로 돌아간 자국들이다.

DAY

01

검증 없는 답은 반쪽이다



수학 지식의 대부분은 지금까지 자연어로 전달되어 왔다. 교과서의 정의도, 논문의 정리도, 칠판 앞의 강의도 모두 비형식의 언어로 쓰여 있다. 기호를 나열하는 듯해도 그 사이사이에는 말로 메워져 있고, 독자는 말로 쓰인 부분을 저마다의 이해로 채운다. 대형 언어 모델(LLM)은 이 언어에 놀랍도록 능숙하다. 그래서 비형식 수학 추론에서 눈에 띄는 성적을 낸다. 그러나 강하다는 말과 완벽하다는 말은 다르다. 추론 사슬 어딘가에는 언제나 근거 없는 비약이 남고, 비형식 추론 안에는 이 비약을 걸러낼 검문소가 없다. 그럴듯하게 읽히는 단계와 옳은 단계 사이의 간격이 사람 눈에만 걸린다.

논문은 여기서 문제를 정면으로 짚는다. 언어 모델을 비형식 추론에만 가두어 두면, 기계가 제공하는 이산적 검증의 기회를 통째로 놓치게 된다는 것이다. 증명 보조기 Lean 4는 명제와 증명을 받아 문법과 논리를 기계가 검사한다. 사람이 눈으로 대조하던 검산을 기계가 기준 없이 일관되게 끝낸다. 증명이 검사를 통과하면 그 옳음은 판독자의 주관과 무관하게 고정된다. 이 검증 신호를 비형식 추론 과정 안으로 끌어들이면 어떻게 될까. 논문이 내놓은 답이 Magenta다. Lean의 신호를 추론의 고리에 이어 붙여, 검증이 실패하면 그 이유를 읽어 다시 추론하는 페루프를 만든다.

Magenta는 학습 없이(training-free) 작동하는 에이전틱 파이프라인이다. 파인튜닝도, 가중치 수정도 없다. 프롬프트와 도구의 조합만으로 자연어로 쓰인 문제 하나를 받아 세 가지 산출물을 잇달아 내놓는다. 답, 그 답을 Lean 4 명제로 옮긴 것, 그리고 기계 검증을 통과시킨 증명. 학습이 없다는 말은 곧 접근법의 성질을 말한다. 검증기와의 결합이 모델 안쪽이 아니라 바깥에서 이루어지므로, 어떤 언어 모델을 꽂아도 같은 파이프라인이 돈다. 이 세 산출물은 채사슬처럼 묶여 있다. 답이 흔들리면 명제가 무너지고, 명제가 무너지면 증명이 무의미해진다. 반대로 증명이 통과하는 순간 앞의 두 고리도 함께 굳어진다. 세 고리 중 하나만 맞는 답은 반쪽이다. 이 장의 제목이 그 뜻이다.

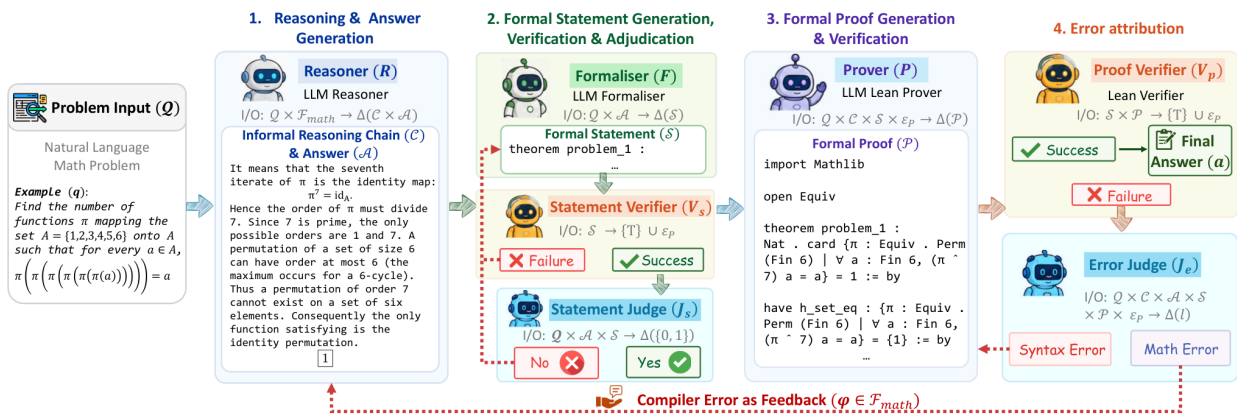


그림 1 Magenta 전체 파이프라인 — 자연어 문제에서 기계 검증 증명까지

그림 1은 가로로 길게 늘린 초와이드 한 장이다. 왼쪽 끝의 자연어 문제에서 출발한 흐름이 오른쪽 끝의 기계 검증 증명까지 한 줄로 달린다. 문제가 추론을 만나 답을 낳고, 답이 명제로 번역되고, 명제에 증명이 붙고, 증명이 판사 앞을 지나며, 필요 하면 흐름이 다시 왼쪽으로 되감긴다. 가로로 길게 늘린 판형은 이 일방통행의 흐름을 강조하기 위한 선택으로 읽힌다. 검증이 실패하는 지점마다 흐름이 되감기는 되돌림 화살표가 아래쪽을 차지한다. 파이프라인 전체의 지형을 한눈에 보여 주는 약도로 삼으면 된다.

그림 왼쪽 끝 문제 입력에서 AIME 2026 문제 하나가 시작된다. 여섯 원소 집합에서 π^6 이 모든 원소를 항등으로 되돌리는 치환의 개수를 묻는 문제다. 초기 추론은 π^7 라는 결론에 도착해 검증에 실패하고, 오류 귀인 판사는 이 실패의 책임이 Lean 문법이 아니라 수학에 있다고 판정한다. 실패 궤적을 물려받은 재도출이 π^6 로 바로잡는다. 부록의 예제가 이 장면의 배경이다. 1부터 6까지 여섯 수의 집합 위에서 치환을 거듭 곱한 결과를 따지는 문제로, 추론은 치환을 순환들의 곱으로 분해하고 각 유형을 하나씩 세어 답 396에 도착한다. 산식은 굳이 여기서 재현하지 않겠다. 요지는 두 가지다. 첫 추론의 지수가 하나 틀렸고, 기계 검증과 판사의 귀인이 그 틀림을 사람 손 없이 고쳤다는 것. 검증이 없었다면 π^7 라는 답이 그럴듯한 서사와 함께 그대로 나왔을 것이다.

이 페루프의 성적은 논문의 도입부에서 이미 확인된다. AIME 2025와 AIME 2026, HMMT 2026 2월 대회로 이루어진 93문제 전부에서 Magenta를 엮은 추론기들이 100퍼센트의 검증 정확도를 기록했고, 오픈 웨이트(open weight) 7B 추론기 K2-Horizon-7B와 결합했을 때는 IMO 2026 여섯 문제 전부를 해냈다. 학습을 하나도 하지 않고 이루어진 성적이라는 점이 이 결과를 더 인상적으로 만든다. 논문의 분석은 또 하나를 일깨운다. 명제 심사가 없으면 기계 검증을 통과한 증명

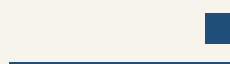
조차 거짓 증명서가 되고, 검증 피드백을 받아 고치는 방식이 무작정 다시 뽑는 방식을 이긴다. 페루프를 이루는 부품이 하나라도 빠지면 완성이 무너진다는 뜻이다.

그림 오른쪽으로 늘어선 직함들이 각각 무슨 일을 하고 무슨 판정을 내리는지가 다음 장의 본론이다. 리즈너(reasoner)와 형식화기, 두 검증기와 증명기, 그리고 두 판사까지 일곱 명의 역할 분담을 하나씩 해부해 보자.

DAY

02

일곱 구성요소로 본 파이프라인



앞 장의 약도를 지도로 삼아 이번에는 건물을 하나씩 들여다본다. Magenta의 몸통은 일곱 구성요소와 특이한 기호 하나로 이루어진다. 논문의 2.1절과 부록 A의 기호표가 이 장의 좌표계다.

실패 마커와 궤적 공간

기호 $\emptyset$ 부터 시작하자. 이 기호는 아직 아무 궤적도 없다는 초기 상태를 뜻하는 동시에, 실패의 자취를 붙잡아 두는 실패 마커이기도 하다. 논문이 정의하는 수학 궤적 공간 $F_{\text{math}} = (C \times A \times S \times P \times E_p) \cup \{\emptyset\}$ 은 다섯 요소의 짝을 한 묶음으로 모은 집합에 $\emptyset$ 를 더한 것이다. 다섯 요소는 추론 맥락 C, 답 A, 명제 S, 증명 P, 증명 오류 E_p 다. 이 데이터 구조의 요지는 재도출에 실패 궤적을 물려주는 일에 있다. 오류 귀인 판사가 실패의 책임을 수학으로 판정할 때마다 그 시행착오의 짝이 이 공간에 기록되고, 리즈너는 다음 라운드에서 이 기록을 함께 읽는다. 처음부터 다시 시작하되 이전에 어디서 넘어졌는지는 기억하는 셈이다. 기억 없는 재시도와 기억 있는 재도출을 가르는 경계가 바로 이 공간이다.

일곱 명의 직무

리즈너(reasoner) R은 문제 q와 궤적 ϕ 를 받아 추론 맥락 c와 답 a의 짝을 확률적으로 뽑아낸다. 부호로 쓰면 $R : Q \times F_{\text{math}} \rightarrow \Delta(C \times A)$ 다. 여기서 $\Delta(X)$ 는 집합 X 위의 확률분포를 뜻한다. 리즈너는 답을 하나 확정하는 기계가 아니라 답 후보를 뽑아내는 확률 분포라는 뜻이다.

형식화기(formaliser) F는 문제 q와 답 a를 받아 Lean 4 명제 s를 샘플한다. $F : Q \times A \rightarrow \Delta(S)$. 번역이 한 번에 옳을 보장이 없으므로 이 단계도 분포다.

구문 검증기(syntactic verifier) V_s 는 명제 s를 Lean의 구문 검사에 통과시킨다. $V_s : S \rightarrow \{T\} \cup E_s$. 통과하면 T를 반환하고, 실패하면 구문 오류 E_s 를 반환한다. $\emptyset$ 도 이 오류 집합에 속한다.

증명기(prover) P는 문제 q와 맥락 c와 명제 s, 그리고 직전 검증의 진단 ϵ 를 받아 증명 π 를 샘플한다. $P : Q \times C \times S \times E_p \rightarrow \Delta(P)$. 진단 자리가 $\emptyset$ 이면 처음부터 증명을 시도하고, 진단이 들어 있으면 그것을 읽고 수리에 들어간다.

증명 검증기(proof verifier) V_p 는 명제 s와 증명 π 의 짝을 Lean 기계 검증에 넣는다. $V_p : S \times P \rightarrow \{T\} \cup E_p$. 통과면 T, 실패면 진단을 돌려준다.

명제 판사(statement judge) J_s 는 문제 q와 답 a와 명제 s를 함께 읽고, 이 명제가 원문제와 답을 보존하는지 심사한다. $J_s : Q \times A \times S \rightarrow \Delta(\{0, 1\})$. 승인이면 1, 기각이면 0을 뽑는다.

오류 귀인 판사(error attribution judge) J_e 는 실패 현장 전체를 읽는다. 문제와 맥락과 답과 명제와 증명과 진단까지 여섯 인자를 받아 $J_e : Q \times C \times A \times S \times P \times E_p \rightarrow \Delta(\{\text{math}, \text{Syntax}\})$. 책임이 수학에 있으면 math, Lean 문법에 있으면 Syntax를 뽑는다.

일곱 명의 직무를 한 표로 정리하면 다음과 같다.

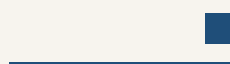
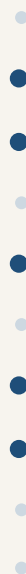
구성요소	하는 일	판정 결과
리즈너 R	문제와 실패 궤적을 읽고 추론 맥락과 답의 짝을 샘플한다	답 후보 (c, a) 제출
형식화기 F	문제와 답을 Lean 4 명제로 번역해 샘플한다	명제 후보 제출
구문 검증기 V_s	명제를 Lean 구문 검사에 통과시킨다	통과면 T, 실패면 구문 오류
증명기 P	명제에 붙일 증명을 진단과 함께 샘플한다	증명 후보 제출
증명 검증기 V_p	명제와 증명의 짝을 기계 검증에 넣는다	통과면 T, 실패면 증명 오류
명제 판사 J_s	명제가 원문제와 답을 보존하는지 심사한다	승인 1 또는 기각 0
오류 귀인 판사 J_e	실패 책임이 수학인지 Lean 문법인지 가른다	math 또는 Syntax

표를 세로로 훑으면 구도가 보인다. 앞의 다섯은 만들고 검사하는 축이고, 뒤의 둘은 심사하고 분기하는 축이다. 만드는 축은 모두 확률 분포에서 샘플을 뽑고, 검사하는 축은 T 또는 오류라는 단정한 판정을 내린다. 샘플의 세계와 판정의 세계가 번갈아 등장하는 이 리듬이 다음 장에서 루프의 골격이 된다. 각 판사가 어떤 기준으로 판정하고 그 판정이 어디까지를 보증하는지는 이어지는 장에서 다루기로 하고, 이 일곱 명이 실제로 어떤 순서로 도는지를 지금부터 따라가 보자.

DAY

03

루프는 이렇게 돈다



일곱 명의 직함을 알았으니 이번에는 경기가 어떤 순서로 진행되는지 본다. Magenta의 알고리즘은 겹친 세 개의 루프다. 부록 A가 알고리즘을 메인 골격과 명제 탐색, 증명 탐색으로 나눠 놓았지만, 겉모습은 하나의 큰 루프 안에 두 개의 작은 루프가 박힌 형태다. 메인 골격은 두 탐색을 부르고, 각 탐색은 자기 결과에 상태 코드를 붙여 돌려준다. 명제 탐색은 승인 또는 기각을, 증명 탐색은 인증 완료, 수학 오류, 수리 소진 세 가지를 반환한다. 의사코드를 나열하는 대신 이 흐름을 처음부터 끝까지 따라가 보자.

바깥 루프에서 증명 인증까지

바깥 루프는 재도출 루프다. 카운터 t 가 0부터 $T-1$ 까지 돈다. 첫 라운드에서 궤적은 $\emptyset$ 다. 리즈너는 실패 기록 없이 문제만 보고 추론 맥락 c 와 답 a 를 뽑는다. 두 번째 라운드부터는 오류 귀인 판사가 남긴 실패 궤적이 궤적 자리에 놓이고, 리즈너는 그것을 읽고 우회 경로를 튼다. 바깥 루프가 한 바퀴 돌 때마다 문제를 푸는 시도가 처음부터 다시 시작되되, 이전 라운드가 어디서 왜 넘어졌는지는 궤적에 남는다.

안쪽의 첫 번째 루프는 명제 루프다. 카운터 m 이 0부터 $M-1$ 까지 돈다. 형식화기가 답 a 를 명제로 번역해 샘플하고, 구문 검증기가 즉시 검사에 넣는다. 검사가 실패하면 그 명제는 폐기하고 다시 샘플한다. 검사를 통과하면 명제 판사가 심사에 나선다. 승인이면 그 명제를 채택하고 명제 루프를 닫는다. 기각이면 역시 폐기하고 다시 샘플한다. 구문 오류든 심사 기각이든 이 루프 안에서는 같은 처벌을 받는다. 버리고 다시 뽑는 것이다. M 번을 다 쓰고도 승인받은 명제가 없으면 명제 탐색은 기각을 반환한다. 이 답을 버려지고, 그 답을 내놓은 추론도 함께 폐기하며, 바깥 루프는 증명 단계를 건너뛰고 다음 라운드로 넘어간다. 증명기는 틀린 답의 명제에 증명을 붙이는 일에 한 번도 시간을 쓰지 않는다.

안쪽의 두 번째 루프는 증명 루프다. 카운터 k 가 0부터 $K-1$ 까지 돈다. 증명기가 명제 s 에 증명 π 를 샘플하는데, 이때 직전 실패의 진단 ϵ 를 함께 받는다. 첫 시도의 진단은 $\emptyset$ 다. 증명 검증기가 τ 를 반환하면 성공이다. 네 짝 (c, a, s, π) 가 인증을 마치고 파이프라인 전체가 이 문제를 끝낸다. 검증이 실패하면 진단 ϵ 를 수집해 오류 귀인 판사로 가져간다. 판정이 Syntax면 진단을 증명기에 쥐여 주고 국소 수리로 재시도한다. 증명만 고치면 되므로 명제와 답은 그대로 돈다. 판정이 math면 이야기가 달라진다. 증명을 아무리 고쳐도 답이 틀렸다는 진단이므로, 네 짝과 진단을 궤적 (c, a, s, π, ϵ) 로 묶어 수학 궤적 공간에 기록하고 바깥 루프로 돌아가 재도출한다. 부록의 증명 탐색은 수리가 끝나기 전 마지막 증명과 마지막 진단을 따로 기억해 두었다가, 수리 예산을 다 쓰고도 성공하지 못하면 수리 소진과 함께 그 마지막 흔적을 돌려준다. 바깥 루프의 예산 T 까지 소진되면 파이프라인은 실패를 반환한다. 인증 완료와 실패, 두 출구만이 이 중첩 루프의 끝이다.

한 문제의 일생을 한 문장으로 요약하면 이렇다. 리즈너가 답을 내고 형식화기가 명제를 만들고 증명기가 증명을 붙이되, 세 곳의 검문소가 통과를 허가할 때까지 구간별로 되감긴다. 구문이 틀리면 명제부터 다시, 수학이 틀리면 추론부터 다시 돈다. 되감기의 반경이 실패의 종류에 따라 세 단계로 나뉜다는 점이 이 파이프라인의 설계적 특징이다.

예산의 크기

원문 부록의 예산 설정이 이 루프의 크기를 고정한다. 재도출 예산은 $T=32$, 명제 샘플 예산은 $M=512$, 증명 시도 예산은 $K=4096$ 이다. 논문이 부록에서 명시적으로 짚는 비대칭이 하나 있다. M 이 K 보다 훨씬 작다. 명제를 다시 뽑는 일보다 증명을 다시 시도하는 일이 훨씬 싸다는 설계 판단이 이 배수에 들어 있다. 증명의 실패는 구문 수준에서 고칠 수 있는 경우가 많지만, 답의 실패는 추론 전체를 다시 해야 하기 때문이다.

증명 검증기의 판정이 내려지는 환경도 짚고 넘어갈 만하다. 바닥에는 SafeVerify라는 인증 포장이 깔려 있다. 증명 회피 토큰과 새 공리의 유입을 원천 차단해, τ 라는 판정이 우연이나 부정행위로 오염되지 않게 한다. 검증 환경 구성의 상세는 실험 설계 장에서 연다.

지금까지 본 것은 루프의 동선이다. 그러나 루프가 어디서 갈라지는지는 판사의 판정이 정한다. Syntax로 갈라지면 수리가 이어지고 math로 갈라지면 재도출이 이어진다. 갈릴길의 판정은 어떤 기준으로 내려지고, τ 판정은 과연 어디까지를 보증하는가. 이 질문에 답하는 일이 다음 장의 본론이다. 루프를 채우는 예산 수치의 근거와 절충은 5장에서 다룬다.

DAY

04

판사 둘과 보증의 경계



루프 안에서 가장 강한 말을 하는 이들은 두 판사다. 둘 다 문장을 만들지 않고 판정만 내리지만, 어느 쪽으로 갈지가 이 판정에 달려 있다. 이 장에서는 두 판사의 역할을 나란히 놓고 대비한 다음, 그 판정이 실제로 무엇을 보증하는지와 어디서 보증이 끝나는지를 짚는다.

두 판사의 역할 대비

명제 판사 J_s 는 성공 후보를 심사한다. 던지는 질문은 하나다. 이 명제가 원문제와 답을 보존하는가. 증명이 아무리 완벽해도 명제가 다른 것을 증명하고 있으면 무용지물이므로, 심사는 증명에 앞서 명제의 충실도를 가린다. 원문제의 조건이 명제에 온전히 남아 있는지, 묻는 것이 같은 것인지, 답이 명제 속에 그대로 살아 있는지를 본다. 오류 귀인 판사 J_e 는 반대로 실패 후보를 심사한다. 질문은 이렇다. 실패의 책임이 수학인가, Lean 문법인가. 같은 검증 실패라도 답이 틀렸을 때와 증명 코드가 문법을 어겼을 때 처방이 정반대이므로, 이 감별이 루프의 분기점이 된다. 한쪽은 통과된 문을 지키고 다른 쪽은 실패의 갈림길에 선다. 둘이 함께 있어야 성공은 위조되지 않고 실패는 낭비되지 않는다.

보증과 그 한계

논문이 밝히는 건전성 보증은 이렇다. 증명 검증기가 τ 를 반환했으면 π 는 s 의 유효한 증명이다. 여기에 명제 판사의 승인, 즉 s 가 문제 q 와 답 a 를 보존한다는 판정이 더해지면 추론 사슬과 답, 명제와 증명의 네 쌍이 q 의 인증된 결과가 된다. 보증이 두 판정의 접점에서 성립한다는 점을 주목하자. 기계 검증만으로는 부족하고, 명제 심사만으로도 부족하다. 검증의 정직함과 명제의 충실함이 한 겹으로 포개질 때 답이 인증된다.

그러나 이 보증에는 경계가 있다. 증명 검증기의 참은 쓰여진 명제에 대해서만 참이다. 검증기는 명제가 진짜 원문제인지 묻지 않는다. 명제 자체가 원문제를 어그러뜨렸다면 그 위의 완벽한 증명은 거짓 증명서(false certificate)를 낳는다. 서명은 진짜인데 증명서가 증명하는 내용이 다른 것이다. 이 경계가 실무적으로 무서운 이유는 위조가 조용하다는 데 있다. 검증은 통과했으므로 시스템은 아무 이상을 보고하지 않고, 거짓 증명서는 정상 출하품처럼 나간다. 그래서 논문은 최종 결과를 소프트 증명서(soft certificate)라 부른다. 형식적으로는 명제에 대해 물리지 않지만, 원래 문제에 대한 뜻은 명제의 충실성만큼일 뿐이라는 한정된 증서다. 이 경계를 실제로 넘어본 사례와 그 숫자는 뒤의 절제 실험 장에서 만나게 된다. 여기서는 한 가지만 남긴다. 심사라는 겹을 걷어내면 기계 검증의 정직함이 명제의 불충실함을 덮는다는 사실이다.

재도출 예산의 직관

마지막 조각은 바깥 루프의 예산에 관한 직관이다. 재도출 한 라운드가 확률 γ 로 답을 고친다고 하자. γ 는 0과 1 사이의 값이다. 문제 q 에 대한 답의 가지수를 $|A(q)|$ 로 쓰면, 필요한 라운드 수 $M(q)$ 는 이 부등식을 따른다.

$$M(q) \leq \log_{1/(1-\gamma)} |A(q)|$$

필요한 라운드 수는 답 가지수의 로그에 불과하다. γ 가 $\frac{1}{2}$ 이면 밑이 2로 떨어져 로그가 익숙한 모양이 된다. 답 후보가 백 가지로 늘어도 라운드는 몇 번만 더 돌면 된다는 뜻이다. 재도출이 비싸 보이는 것은 한 라운드의 몸집 때문이지, 필요한 라운드 수 때문이 아니다. 실패를 기억하며 다시 추론하는 일이 결코 폭발적으로 비대해지지 않는다는 계산이 바깥 루프에 예산 $T=32$ 를 실어 보낼 근거가 된다.

방법의 정독은 여기서 끝난다. 일곱 구성요소와 세 겹의 루프, 두 판사의 판정과 보증의 경계까지가 Magenta의 설계 전부다. 다음 장부터는 이 설계를 실험대에 올린다. 어떤 모델을 어떤 예산으로 돌렸는지가 첫 안건이다.

DAY

05

실험 설계 — 모델, 벤치마크, 예산



방법을 다 봤으니 이제 숫자를 본다. 그런데 표를 읽기 전에 실험이 어떻게 짜였는지 알아야 그 숫자가 무엇을 말하는지 안다. 누구를 리즈너로 세웠고, 어떤 문제를 냈고, 누가 증명을 쓰고, 무엇이 검증을 맡았는지, 그리고 얼마나 많은 시도를 허용했는지. 이 장에서는 다섯 축을 모두 짚는다. 결과의 신뢰도는 설계가 먼저 결정하기 때문이다.

리즈너 일곱, 두 진영

Magenta는 자체 두뇌를 갖지 않는다. 추론기(reasoner) 자리에 어떤 모델을 앉히느냐가 곧 시스템의 성능을 정한다. 논문은 일곱 모델을 추론기로 세웠다. Claude Opus 5, Gemini 3.7 Flash, Kimi K3, K2-Horizon-7B, K2-Horizon-375B, Qwen3.8-27B, GPT-5.6-Sol(Codex)이 그 목록이다.

이 명단에는 구도가 있다. 앞쪽 셋은 무게를 공개하지 않는 프린티어 진영이고, K2-Horizon 두 모델과 Qwen3.8-27B는 누구나 내려받아 돌릴 수 있는 오픈 웨이트 진영이다. GPT-5.6-Sol(Codex)은 코딩 에이전트 형태로 붙은 별개의 축이다. 같은 파이프라인을 일곱 두뇌에 얹어 보면, 성능 격차가 모델에서 오는지 파이프라인에서 오는지 가려진다. 작은 오픈 웨이트 모델이 어디까지 따라오는지가 이 장의 절반은 긴장점이다.

93문제와 6문제

벤치마크는 AIME 2025, AIME 2026, HMMT Feb 2026 세 종류이고 합계 93문제다. 여기에 IMO 2026 여섯 문제를 얹는다. AIME와 HMMT는 정답을 내놓는 대회이고, IMO는 증명을 완성해야 하는 대회라서 난도의 성격부터 다르다. 93문제에서 전부 통과하면 정확도 100.00이고, IMO 6문제는 그 위에 얹는 최고 난도의 시험이다.

증명기는 내부와 외부

증명기(prover) 자리에서도 비교 축이 하나 더 놓인다. 한쪽은 논문 자체의 Leanstrat이고, 다른 쪽은 외부 에이전트 Codex다. 같은 문제와 같은 명제를 주고 두 증명기가 각각 어떻게 도는지 따져 보면, 성공률과 비용의 그림이 뚜렷하게 갈린다. 그 대비가 어떻게 펼쳐지는지는 뒤의 그림 장에서 다룬다.

검증기에 심은 정직성

검증 환경은 Lean v4.29.1과 Mathlib 위에 SafeVerify를 얹은 것이다. SafeVerify는 금지 목록을 갖는 검증기다. `sorry`와 `admit`, 신규 공리 선언, `native_decide`가 증명에 끼어 들면 통과를 인정하지 않는다. 증명 보조기는 원래 기계가 옳음을 따지는 도구지만, 증명자가 요행의 틈을 파고들면 검증기가 그 요행을 그대로 통과시켜 버릴 수 있다. 금지 목록은 바로 그 틈을 막는 장치다. 요점은 단순하다. 검증기 자체에 정직성 정책을 심는다는 것이다. 모델이 정직하길 바라는 대신, 부정직한 증명이 물리적으로 통과하지 못하는 환경을 만든 것이다.

예산과 샘플러

마지막 축은 시도 예산이다. 설정을 적어 본다.

- 추론 재시도 T는 32, 명제 샘플 M은 512, 증명 시도 K는 4096이다.
- 리즈너 샘플러는 temperature 0.6, top_p 0.95, seed 42로 고정했고, 컨텍스트는 524,288토큰이다.
- 리즈너 최대 출력은 64,000토큰이고, IMO에서만 128,000토큰으로 올랐다.
- Leanstrat은 요청당 최대 출력이 80,000토큰이고 실험 래퍼에서는 32,000토큰으로 잘렸다. 궤적 하나가 누적 4,000,000토큰에 다다르면 168,000토큰 기준으로 컴팩션한다.

여기서 노력의 비대칭이 읽힌다. 리즈너에게는 32번의 추론 기회를 주지만, 증명기에게는 4,096번의 시도를 연다. 어려운 문제에서는 증명이 추론보다 훨씬 자주 실패한다는 사실이 예산에 그대로 새겨져 있는 셈이다.

설계는 이것으로 끝이다. 요약하면 일곱 리즈너, 93문제와 6문제, 내부와 외부 증명기, 정직성을 심은 검증기, 그리고 증명 쪽으로 기운 예산이다. 다음 장에서는 이 설계가 내놓은 표를 그대로 펼친다. 베이스라인의 천장과 Magenta 결합의 전면 통과가 같은 판 위에서 어떻게 대비되는지 숫자로 확인한다.

DAY

06

표로 보는 완승



결과를 말로 꾸미면 과장으로 들리기 쉽다. 그래서 이 장은 표 두 개로 승부한다. 첫 번째 표는 Magenta 없이 달린 베이스라인 성적이고, 두 번째 표는 같은 모델에 Magenta를 결합한 성적이다. 두 표를 나란히 놓으면 이 논문의 주장이 한눈에 담긴다.

베이스라인의 천장

표 6-1이 Magenta 없이 달린 성적이다. 판은 앞 장에서 본 대로 AIME 2025, AIME 2026, HMMT Feb 2026, 총 93문제다.

표 6-1 베이스라인 정확도(단위 퍼센트)

모델	AIME 2025	AIME 2026	HMMT Feb 2026	Overall
Claude Opus 5	100.00	100.00	96.97	98.92
Gemini 3.7 Flash	100.00	100.00	90.91	96.77
Kimi K3	93.33	90.00	78.79	87.10
K2-Horizon-7B	83.33	80.00	60.61	74.19
K2-Horizon-375B	93.33	90.00	72.73	84.95

표를 세로로 훑어 보면 천장이 보인다. 사전학습만으로 가장 강하다는 Claude Opus 5는 AIME 두 해에서 만점을 받았지만 93문제 전체를 잇는 만점은 없었고, 가장 약한 칸이 HMMT의 96.97이다. AIME 2025에서 만점에 닿은 모델은 표 안에서 돌뿐이고, 그마저 Gemini 3.7 Flash는 HMMT와 Overall을 합쳐 세 문제를 남긴다. 여기에 표에 없는 두 리즈너의 단독 성적을 덧붙이면 바닥이 더 선명해진다. Qwen3.8-27B는 100.00, 86.67, 87.88, Overall 91.40이고, GPT-5.6-Sol(Codex)은 90.00, 86.67, 78.79, Overall 84.95다. AIME 2025 만점 행렬에 Qwen3.8-27B까지 더하면 셋이 되지만, 93문제 전체를 채우는 모델은 이번에도 없었다. 이것이 Magenta가 넘어야 할 출발선이다.

Magenta 결합, 전면 100.00

표 6-2는 같은 판에 Magenta를 결합한 성적이다. 네 행이 모두 같은 값을 가리킨다.

표 6-2 Magenta 결합 정확도(단위 퍼센트)

모델	AIME 2025	AIME 2026	HMMT Feb 2026	Overall
K2-Horizon-7B+Magenta	100.00	100.00	100.00	100.00
K2-Horizon-375B+Magenta	100.00	100.00	100.00	100.00
Qwen3.8-27B+Magenta	100.00	100.00	100.00	100.00
GPT-5.6-Sol(Codex)+Magenta	100.00	100.00	100.00	100.00

네 모델이 93문제 전부를 통과했다. 단순히 만점을 한 것이 아니라, 서로 다른 네 두뇌에서 같은 만점이 재현됐다는 점이 무게를 더한다. 상승폭을 행별로 읽으면 등반의 결이 드러난다. K2-Horizon-7B는 AIME 2025에서 16.67포인트, AIME 2026에서 20.00포인트, HMMT에서 39.39포인트, Overall에서 25.81포인트 올랐다. K2-Horizon-375B는 6.67, 10.00, 27.27, 15.05포인트다. Qwen3.8-27B는 0.00, 13.33, 12.12, 8.60포인트, GPT-5.6-Sol(Codex)은 10.00, 13.33, 21.21, 15.05포인트다.

이 표의 핵심 대목은 HMMT의 7B 행이다. HMMT에서 단독 60.61에 그쳤던 모델이 결합 후 만점에 닿았고 폭이 39.39포인트에 이른다. 반대로 이미 100.00이었던 Qwen3.8-27B의 AIME 2025는 올라갈 곳이 없어 상승이 0.00포인트다. 출발점이 낮을수록 폭이 크다는 산술이 여기서 자연스럽게 드러난다. 그리고 그 폭을 만든 것은 학습이 아니다. 파라미터는 한 자루도 바꾸지 않았다. 검증 루프만으로 각 모델의 천장을 끝까지 올린 것이다.

두 표를 겹쳐 놓으면 논문의 주장이 절로 정리된다. 베이스라인의 천장은 모델마다 높이가 달랐지만, Magenta 결합의 천장은 전부 같은 높이에서 맞닿는다. 다음 장에서는 이 완승의 가장 높은 단을 들여다본다. 오픈 웨이트 7B 모델이 IMO 무대에서 증명서를 받는 장면이다.

DAY

07

7B가 IMO를 넘었다



앞 장의 표가 AIME와 HMMT의 완승이었다면, 이 장은 그 위에 얹은 최고 난도의 시험을 본다. IMO 2026 여섯 문제. 오픈 웨이트 진영의 소형 모델이 이 무대에서 기계가 검증한 증명서를 받았다는 사실이 이 장의 결론이다.

여섯 문제, 여섯 증명서

K2-Horizon-7B를 추론기로 세운 채 Magenta를 돌리자, IMO 2026 여섯 문제 전부에서 Lean이 받아들이는 증명이 완성됐다. 프런티어 모델의 전유물로 여겨져 온 무대에서 이름의 규모가 가장 작은 모델이 증명서를 받아 든 것이다. Magenta가 학습 없이 검증 루프만으로 만든 성과라는 점을 상기하면, 이 결과는 모델을 키우지 않고 난도를 넘는 길이 실제로 있음을 보여 준다. 물론 여섯 문제가 고르게 풀린 것은 아니다. 문제마다 루프가 도는 횟수가 크게 달랐고, 그 편차가 다음 절의 주제다.

자기수정의 편차

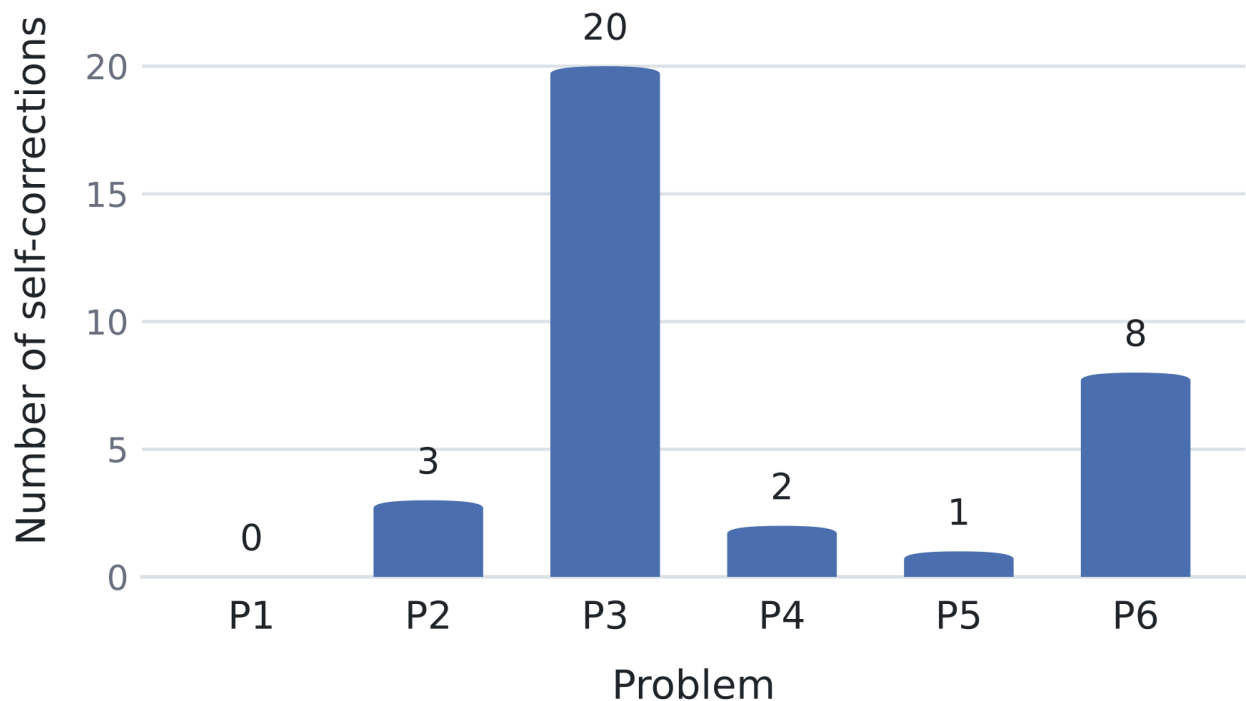


그림 2 IMO 2026 문제별 자기수정 횟수

그림 2는 여섯 문제 각각에서 자기수정이 몇 번 일어났는지 보여 준다. 값은 차례로 0회, 3회, 20회, 2회, 1회, 8회이고 평균은 5.7회다. 최댓값과 최솟값의 격차가 20에 이른다. 어떤 문제는 처음 세운 증명이 한 번도 틀리지 않았고, 어떤 문제는 스무 번을 고쳐야 겨우 통과했다. 루프가 길게 도는 풍경이 곳곳에서 벌어진 셈이다. 같은 시스템, 같은 예산을 놓고 봐도 문제의 성격이 수정량을 결정한다는 사실이 이 그림의 요점이다.

IMO가 아니라 정답식인 AIME 2026에서 같은 지표를 보면 평균 자기수정이 K2-Horizon-7B는 5회, K2-Horizon-375B는 2회다. 작은 모델이 더 많이 돈다. 이 격차는 그림 2의 편차와 같은 방향을 가리킨다. 정제 라운드 표로 옮기면 그 방향이 더 뚜렷해진다.

정제 라운드의 대조

정제 라운드는 검증 실패 이후 증명이 고쳐지는 횟수를 센 지표이고, 초기 시도를 1로 계산한다. 표 7-1은 원문 부록의 정제 라운드 표를 옮긴 것이다.

표 7-1 모델·데이터셋별 정제 라운드(평균/최대, 초기 시도 1 포함)

데이터셋	7B 평균	7B 최대	375B 평균	375B 최대
AIME 2025	1	3	1	1
AIME 2026	2	5	1	2
HMMT 2026	3	10	1	2

표를 두 진영으로 나눠 읽으면 대비가 선명하다. K2-Horizon-375B는 어느 데이터셋에서도 평균 1회, 최대 2회 안에 머문다. 반면 K2-Horizon-7B는 AIME 2025에서 평균 1회와 최대 3회로 시작해, AIME 2026에서 평균 2회와 최대 5회, HMMT 2026에서 평균 3회와 최대 10회로 다섯 배까지 벌어진다. 같은 문제를 같은 시스템으로 풀되, 두부의 크기가 반 복 횟수를 대신 내는 그림이다. 앞 절의 AIME 평균 5회와 2회가 이 표에서 같은 방향으로 확인되는 셈이다. 수정이 공짜는 아니므로, 이제 비용의 척도인 토큰으로 눈을 옮긴다.

토큰으로 본 IMO

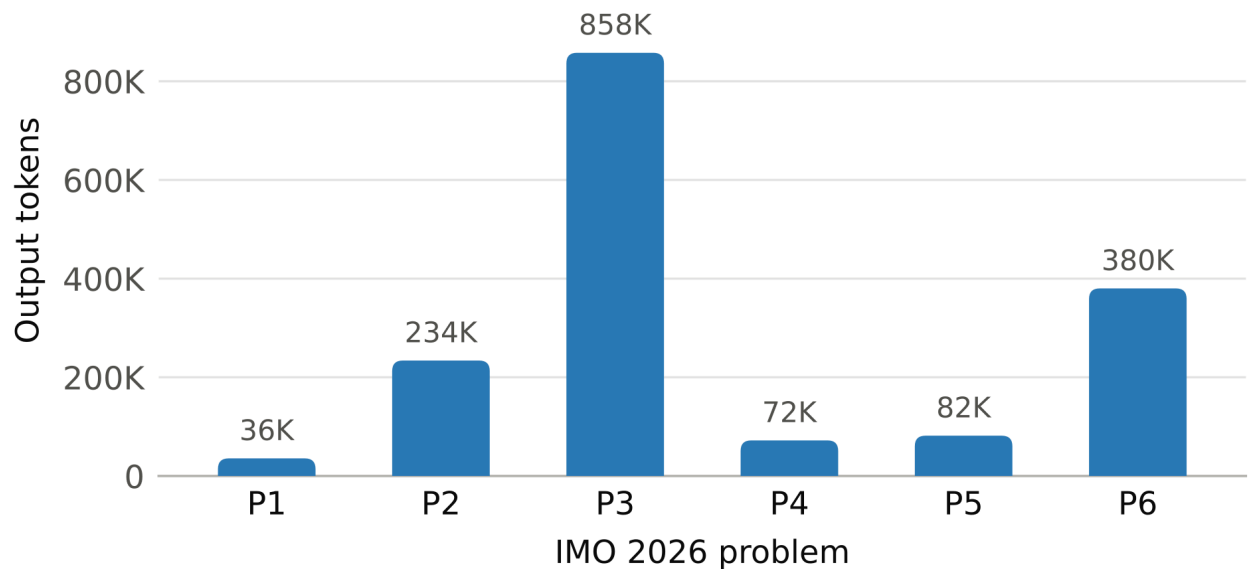


그림 6 IMO 2026 솔루션 토큰 사용량

그림 6은 IMO 솔루션이 소비한 리즈너 토큰을 문제별로 보여 준다. 실험 설계 장에서 IMO 출력 상향을 적어 두었던 사실이 여기서 의미를 얻는다. IMO에서만 리즈너 최대 출력을 64,000토큰에서 128,000토큰으로 올렸다. 그림 2에서 봤듯 스무 번을 고쳐야 하는 문제가 있었으니, 한 번의 응답에 담을 사고의 길이를 두 배로 늘린 것은 긴 루프를 감당하기 위한 조치였다고 읽힌다. 실제로 여섯 문제 가운데 문제 3과 문제 6이 압도적으로 큰 예산을 썼고, 이는 자기수정이 각각 20회와 8회로 가장 많았던 두 문제와 정확히 겹친다. 수정이 많은 문제일수록 토큰이 크게 불어나는 건 자연스러운 귀결이고, 그림이 그 폭을 고스란히 보여 준다.

부록에 실린 여섯 증명

원문 부록에는 여섯 문제 전체의 Lean 인증 풀이가 그대로 수록돼 있다. 본문에서는 그 성격만 소개하고 마친다. 치환을 다루는 문제, 삼각형과 여러 점의 거리 등식을 확정하는 기하 문제, 함수방정식 등이 섞여 있었고, 풀이마다 대수적 조작과 구조 파악의 비중이 달랐다. 어떤 풀이가 어떤 수정 횟수를 냈는지 번호로 짝지어 단정하지 않는 것은, 원문이 번호와 주제를 나란히 세워 두지 않기 때문이다. 수치로 확실한 것은 그림 2의 수정 횟수와 표 7-1의 정제 라운드뿐이다.

여기까지가 완승의 보고다. 다만 이 성과가 공짜였던 것은 아니다. 다음 장에서는 완승을 떠받친 비용의 이야기를 연다. 명제가 어떻게 만들어지고 얼마나 갈려 나가는지, 명제 생성의 경제학을 들여다본다.

DAY

08

명제 생성의 경제학



증명기가 아무리 많은 시도를 허용해도, 그 앞에 형식 명제가 없으면 시작조차 없다. Magenta의 비용은 증명 단계보다 한 발 앞인 명제 생성 단계에서 먼저 걸린다. 이 장은 자연어 문제를 Lean 명제로 바꾸는 형식화 단계를 경제학의 눈으로 읽는다. 누가 몇 번의 호출로 커버리지를 만드는지, 반복 호출은 커버리지를 어떻게 사들이는지, 그리고 서로 다른 두 시스템을 공정하게 재는 잣대는 무엇인지를 차례로 본다.

형식화기의 값어치는 호출당 성공률로 갈린다

같은 추론기에 서로 다른 형식화기(formaliser)를 얹으면 필요한 호출 수가 수십 분의 일로 변한다. 원문은 두 형식화기를 나란히 돌렸다. 한 구성은 Codex를 형식화에 썼고, 다른 구성은 형식화 전용으로 학습된 Goedel을 썼다. Codex는 앞에서 바깥 증명기 자리로 만난 모델이지만, 증명 뒷자리와 별개로 형식화 자리에서도 최고 성적을 낸다. 첫 호출에서 Lean이 그대로 받아들이는 명제를 뽑아낸 비율은 Codex가 약 66퍼센트, Goedel이 42퍼센트였다. 두 값의 차이만 놓고 보면 우열이 크지 않다. 그러나 이 차이는 더해지지 않고 곱해진다. 문제 하나를 형식 명제로 옮기는 시도는 서로 독립에 가깝고, 호출당 성공률이 낮을수록 같은 커버리지를 채우는 데 필요한 호출이 지수적으로 불어난다. 실측이 이 논리를 그대로 확인한다. 문제 전부를 형식화하는 데 Codex는 여섯 호출로 족했지만, Goedel은 약 120호출을 요구했다. 여섯 호출 시점의 Goedel 누적 커버리지는 약 68퍼센트로, 같은 지점에서 Codex는 이미 전체를 덮고 난 뒤였다.

여기서 얻는 교훈은 절약이 아니다. 예산이 정해져 있을 때 호출당 성공률이 낮은 형식화기는 꼬리를 자른다. 남은 예산으로 끝내 못 옮긴 문제는 형식 명제 자체가 없고, 명제가 없으면 증명기는 그 문제에 대해 단 한 번의 시도 기회도 얻지 못한다. 컴퓨팅이 모자라 틀린 것이 아니라, 형식화 단계에서 그 문제의 커버리지가 통째로 증발한 것이다. 호출당 성공률의 차이는 결국 잃어버린 문제의 수로 정산된다.

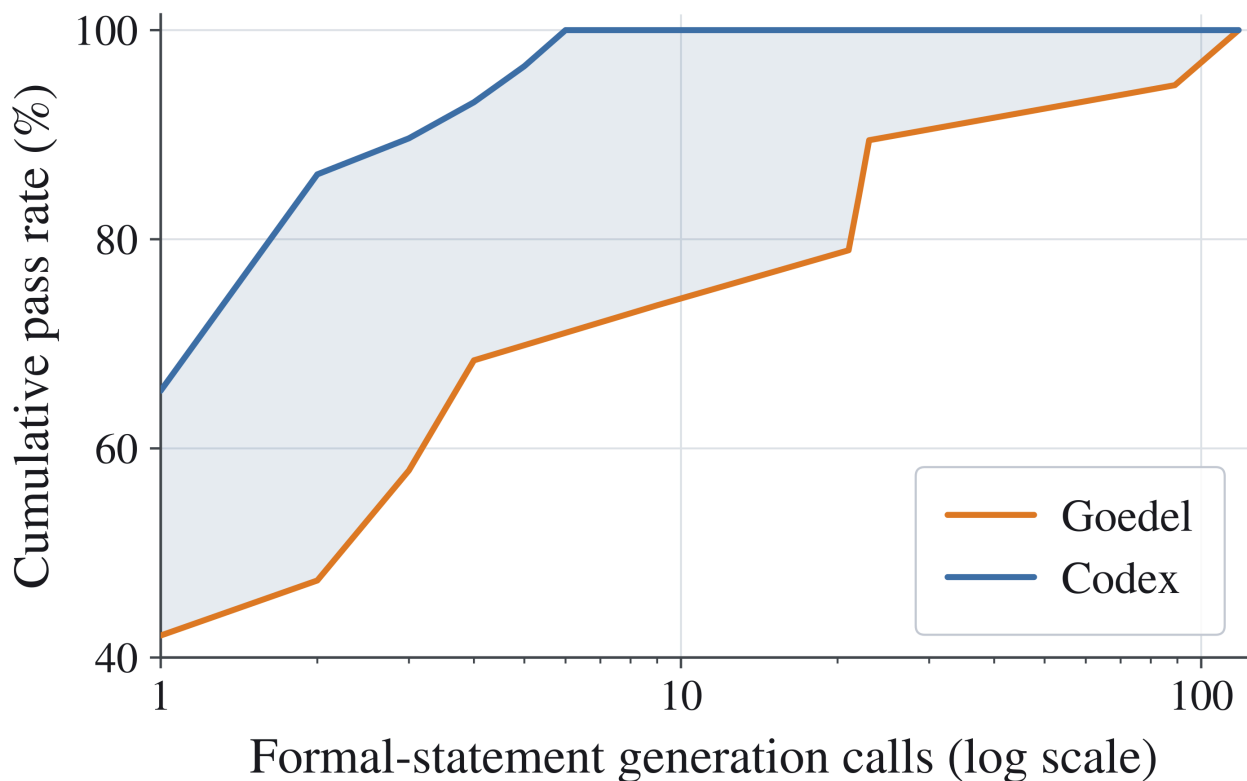


그림 3 명제 생성 호출 누적과 통과율 — Codex vs Goedel

그림 3은 호출을 하나씩 쌓아 가면서 명제 통과율이 어떻게 오르는지 두 형식화기를 겹쳐 그린 곡선이다. Codex 곡선은 몇 번 만에 천장에 닿고, Goedel 곡선은 완만한 기울기로 훨씬 길게 이어진다. 두 곡선 사이의 가로 간격이 곧 비용 차이이고, 같은 세로선에서의 높이 차이가 곧 그 시점까지 읽은 요령은 끝점에 두지 말고 기울기에 뒤야 한다. 초반 기울기가 가파를수록 예산 대비 커버리지가 싸다는 뜻이기 때문이다.

내부 호출로 커버리지를 사들이는 방법

Magenta가 실제로 채택한 형식화기는 Leanstral이다. 이 구성은 한 번의 호출로 만족하지 않고 명제 샘플링을 반복한다. 같은 문제와 답에 대해 명제 후보를 M 회 뽑고, 그중 구문 검사와 명제 판사를 통과한 것이 하나라도 나오면 그 문제는 형식화에 성공한 것으로 친다. 커버리지가 단일 호출의 재능에서 나오지 않고 축적에서 나오는 구조다. 실패한 후보는 버려지지 만 시도 자체는 남아, 예산을 커버리지로 바꾸는 환율을 시스템이 스스로 조정한다.

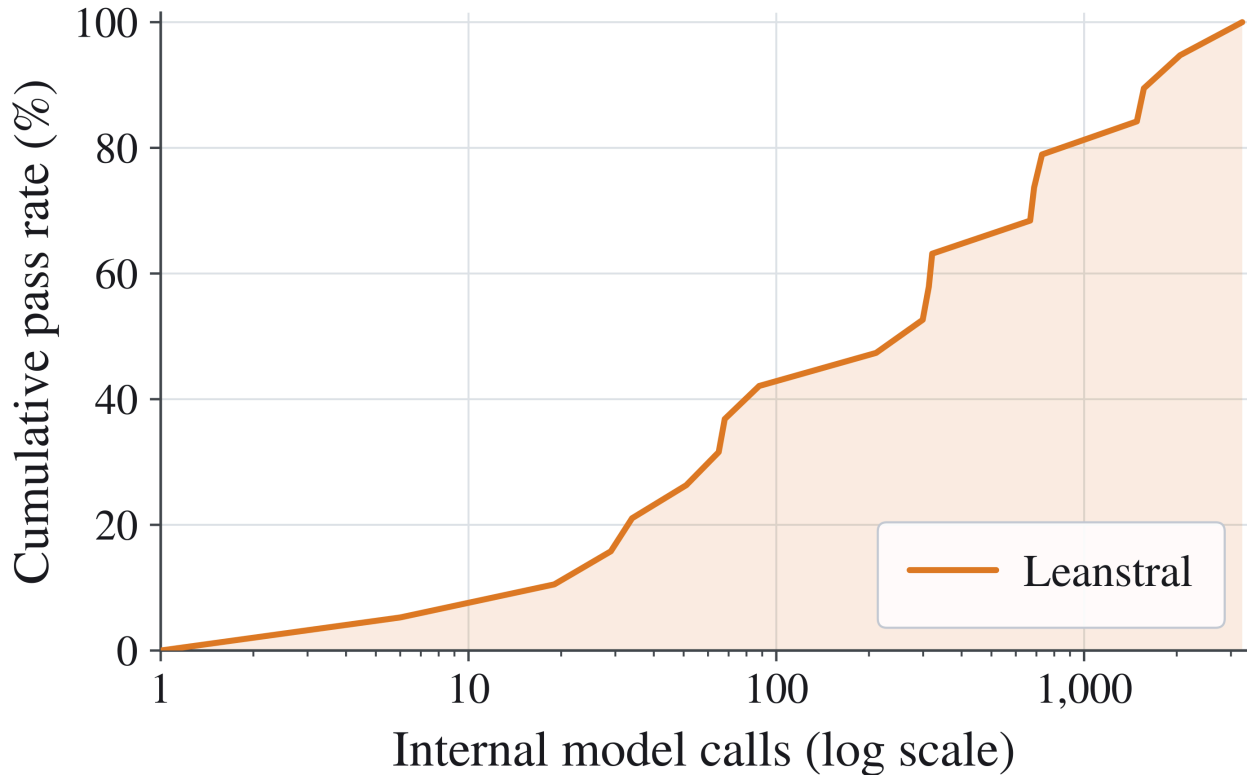


그림 4 Leanstral 내부 호출과 커버리지

그림 4는 이 축적의 실제 가격표다. 문제의 절반을 넘으려면 Leanstral의 내부 호출, 곧 명제 확인과 증명 시도, 수리를 아우르는 호출이 대략 3×10^2 에 이르고, 가장 까다로운 문제까지 모두 덮는 데는 최대 3×10^3 까지 불어난다. 중앙값과 최댓값의 간격이 넓다는 사실이 중요하다. 평균적 문제는 싸게 처리되지만, 꼬리 문제 하나가 전체 내부 예산을 집어삼킬 수 있다. 그래서 이 호출 예산은 명제 샘플링만의 몫이 아니라 파이프라인 전체의 몫으로 읽어야 하고, 꼬리가 끌어당기는 비용을 별도 항목으로 떼어 봐야 한다.

예산을 조건으로 붙인 통과율

호출 수만 나란히 놓으면 반쪽짜리 비교다. 예산 구조가 다른 두 시스템을 공정하게 재려면 같은 예산에서 누가 더 많이 검증에 성공하는지 물어야 한다. 원문은 이를 위해 예산 조건부 통과율(budget-conditioned pass rate)을 정의한다.

$$\text{Pass}_u(\beta) = \frac{1}{|\mathcal{D}|} \sum_{q \in \mathcal{D}} \mathbb{1}[\text{Ver}(q) = 1 \wedge u(q) \leq \beta]$$

식의 뜻은 단순하다. 문제 집합 $\mathcal{D}$ 의 각 문제 q 에 대해, 검증이 통과했고 그때까지 소비한 예산이 상한 β 이하일 때만 한 문제로 셈한다. 이 지표를 β 를 바꿔 가며 그리면 예산 대비 성과 곡선이 나온다. 같은 β 에서 누가 더 높은가를 묻는 순간, 예산이 많아서 이겼다는 변명은 성립하지 않는다. 지금까지 본 것은 명제 단계의 경제학이다. 명제가 준비되면 비용은 증명 단계에서 새로 청구된다. 다음 장은 그 비용 곡선을 네 개의 패널로 나눠 읽는다.

DAY

09

비용 곡선의 네 패널



앞 장에서 명제 단계의 호출 경제를 봤다. 명제가 준비되면 비용은 증명 단계에서 청구되고, 이번에는 청구 방식이 두 갈래로 나뉜다. 한 구성은 Codex를 바깥 증명기로 두고 실패할 때마다 문제를 다시 추론하는 외부 루프를 돌린다. 다른 구성은 Leanstrat의 내부 루프 안에서 재시도를 흡수한다. 같은 문제, 같은 목표, 다른 루프 설계. 원문은 이 대결을 네 개의 패널로 나눠 그렸고, 패널마다 비용이 갈리는 지점이 다르다.

패널 (a) 총 호출과 해결률

첫 패널은 모든 호출을 합산한 총 호출 기준의 통과율이다. 이 축에서 유리한 쪽은 Codex다. 같은 통과율에 도달하는 데 필요한 호출이 더 적고, 모든 문제를 덮는 시점도 더 이르다. 겉보기와 반대되는 결과지만 계산의 포인트는 명확하다. Leanstrat 구성이 아낀 것은 바깥 비용이지 총 노력이 아니다. 아낀 외부 호출은 내부 루프에서 몇 배의 호출로 되갚아지고, 그 내부 호출까지 전부 세면 그림의 순서가 뒤집힌다. 패널 (a)는 바로 그 뒤집힌 그림이다.

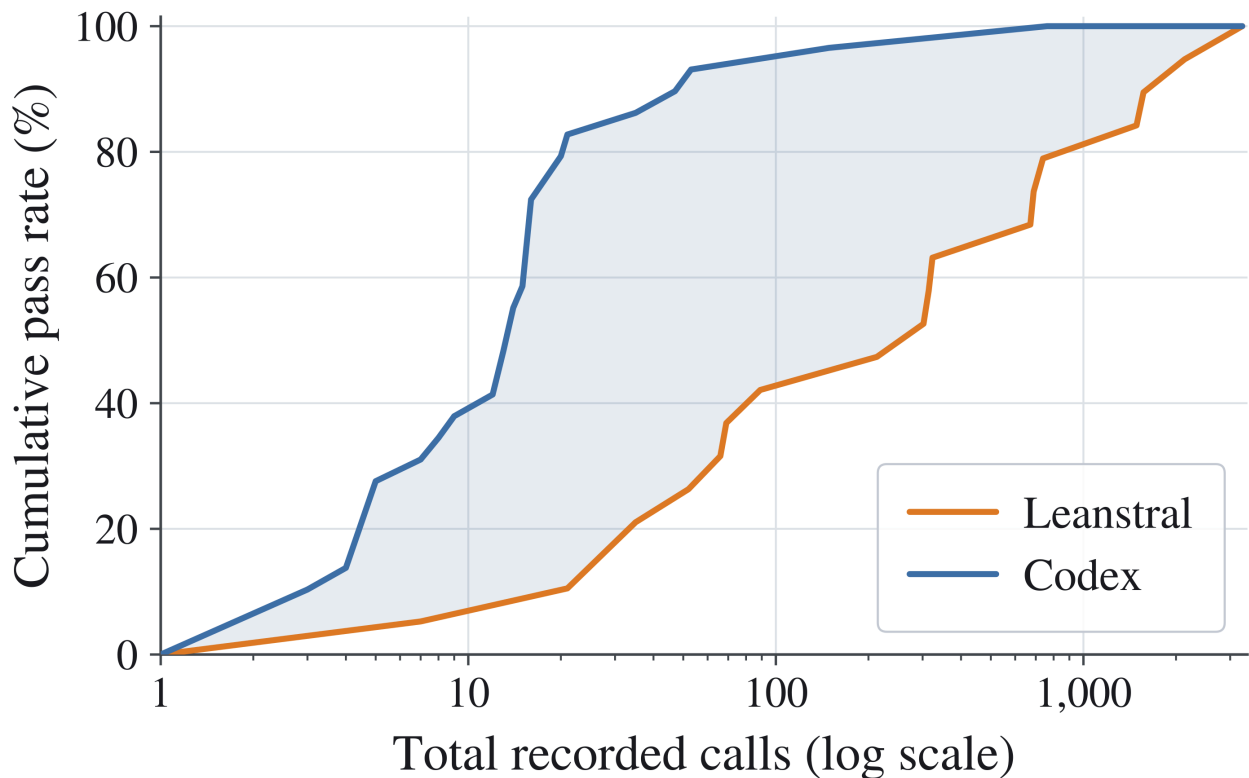


그림 5(a) 총 호출과 해결률

그림 5(a)의 가로축은 총 호출, 세로축은 통과율이다. Codex 곡선이 왼쪽 위에 자리하고, Leanstrat 곡선은 오른쪽 아래로 밀려 있다. 같은 높이에 오르려면 Leanstrat이 훨씬 더 오른쪽으로 나아가야 한다는 뜻이다. 총 호출만으로 시스템을 재면 Leanstrat이 불리해 보인다. 그 불리의 정체를 가르려면 실패가 어디서 끝나는지를 봐야 한다. 다음 패널의 주제다.

패널 (b) 외부 실패 라운드

둘째 패널은 실패의 행선지를 본다. 증명 실패가 오류 귀인 판사, 곧 실패를 수학 문제와 구현 문제로 갈라 라우팅하는 판사에 도달한 비율이 Leanstrat 구성은 58퍼센트, Codex 구성은 83퍼센트였다. Codex는 실패 자체도 많고, 생긴 실패가 라우터까지 올라가 문제를 처음부터 다시 추론하는 재도출로 번지는 비중이 크다. 이 그림에는 축 위의 한 점이 더 읽힌다. 가로축이 0인 지점, 즉 외부 실패 없이 끝난 문제가 Leanstrat 42퍼센트, Codex 17퍼센트다. 재도출로 튕겨 나가지 않고 내부 루프 안에서 말끔히 끝나는 문제가 두 배 이상 많다는 뜻이다. 실패가 내부에서 끝나지 못하고 바깥 루프로 새는 순간, 비용은 증명기 한 대의 몫을 넘어 추론기 전체의 몫으로 불어난다.

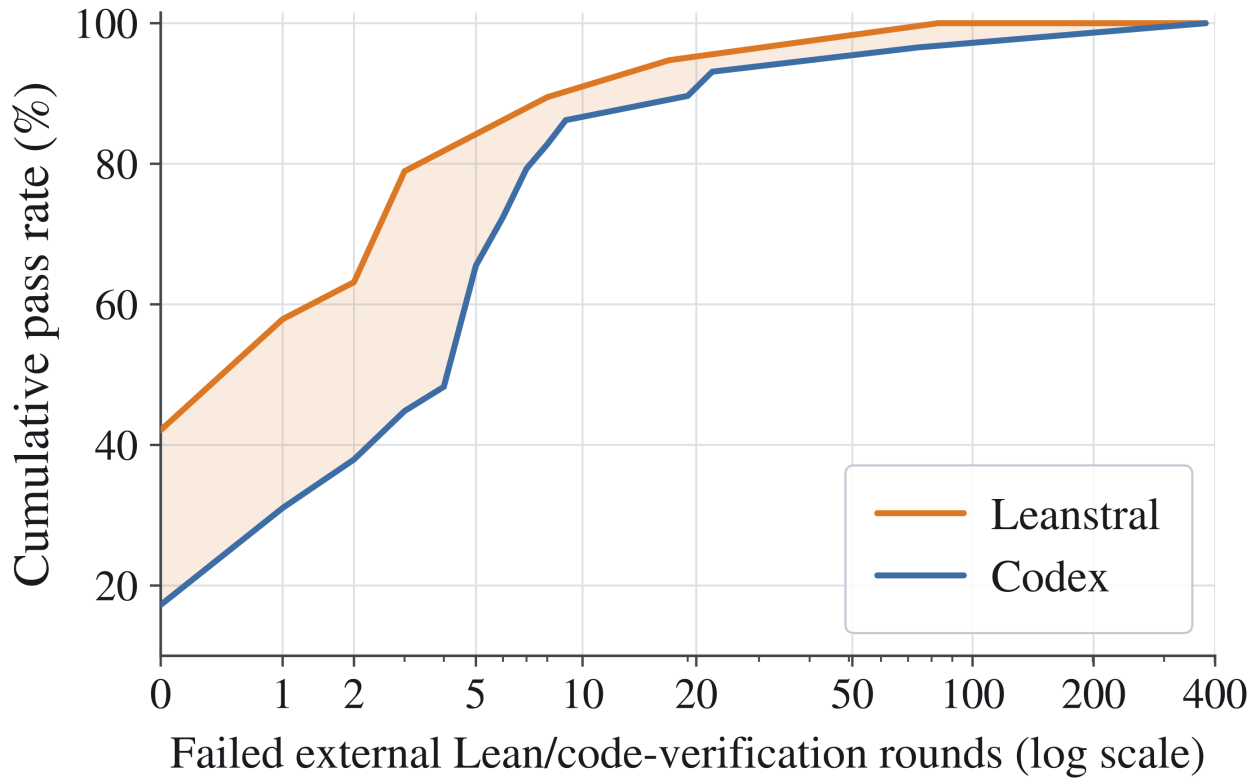


그림 5(b) 외부 실패 라운드 분포

그림 5(b)는 가로축에 외부 실패 라운드를 두고 누적 통과율을 그린 곡선이다. 가로축이 0인 절편에서 Leanstrai 곡선이 Codex 곡선보다 훨씬 높게 출발하고, 라운드가 쌓일수록 두 곡선의 간격이 곧 실패가 라우터에 도달해 재도출로 번지는 비용이 된다. 패널 (a)의 순위는 여기서 뒤집힌다. 총 호출은 많지만 실패를 안쪽에서 흡수하는 구성이 바깥 비용에서는 이긴다.

패널 (c) 리즈너 토큰

셋째 패널은 추론기(reasoner)가 실제로 소비한 토큰을 잔다. 전체 커버리지, 곧 모든 문제를 덮을 때까지의 리즈너 토큰 사용량이 Leanstrai 구성은 약 6×10^4 , Codex 구성은 약 9×10^5 였다. 호출 횟수의 차이가 그대로 토큰 청구서에 나타난다. 바깥 루프가 도는 만큼 추론기를 다시 호출해 문제를 새로 읽고 새로 추론해야 하기 때문이다.

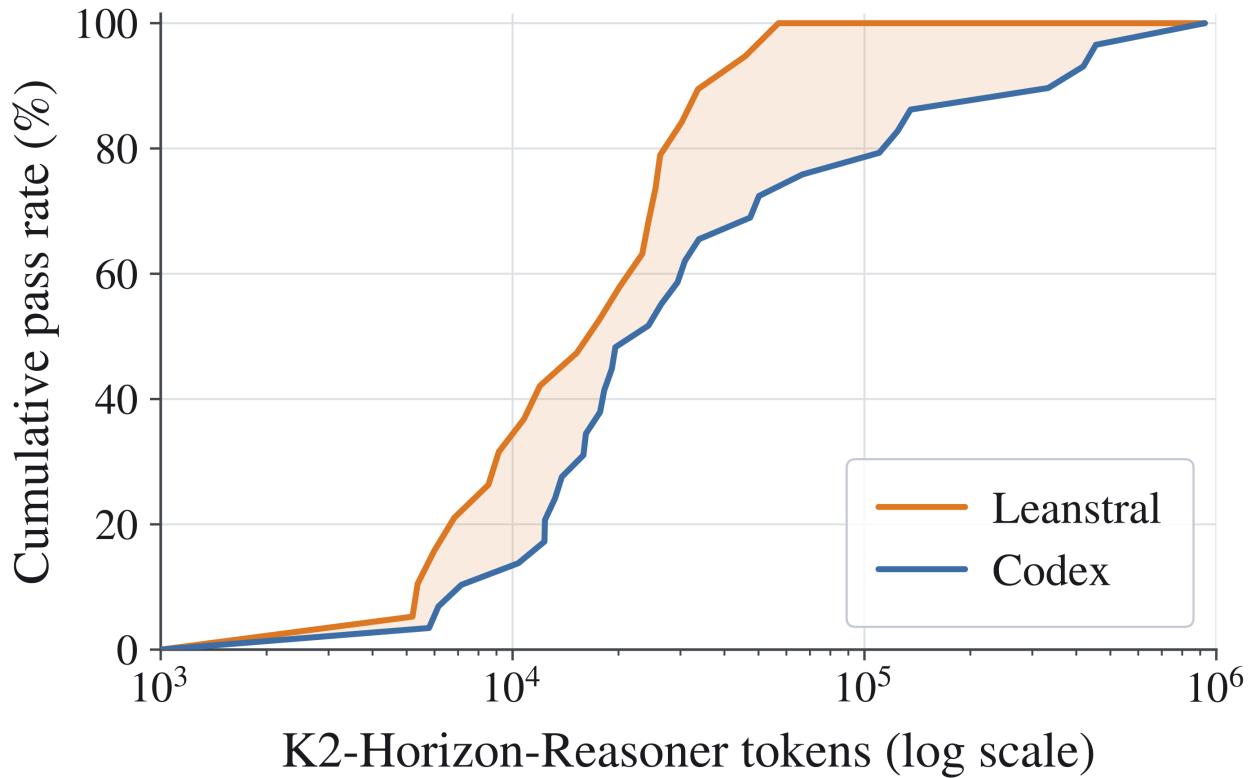


그림 5(c) 리즈너 토큰 사용량

그림 5(c)는 로그 눈금에서 두 구성의 토큰 소비를 나란히 놓는다. 두 곡선의 가로 간격이 한 자릿수 이상 벌어져 있다는 사실만으로 충분하다. 같은 일을 하는 두 설계가 청구서에서는 등급이 다른 고객처럼 보인다는 것, 그리고 그 격차가 능력이 아니라 루프의 위치에서 나온다는 것이 이 패널의 요지다.

패널 (d) 검증된 Lean 코드 길이

넷째 패널은 산출물의 크기를 잴다. 검증을 통과한 증명의 길이가 Leanstral 구성은 약 1.1×10^4 토큰, Codex 구성은 약 2×10^4 토큰으로 절반 가량 차이가 난다. 성공한 증명조차 외부 루프 설계에서는 더 길어진다. 재도출이 반복되는 동안 증명에 군더더기가 끼어들고, 검증은 통과해도 코드는 비대해진다.

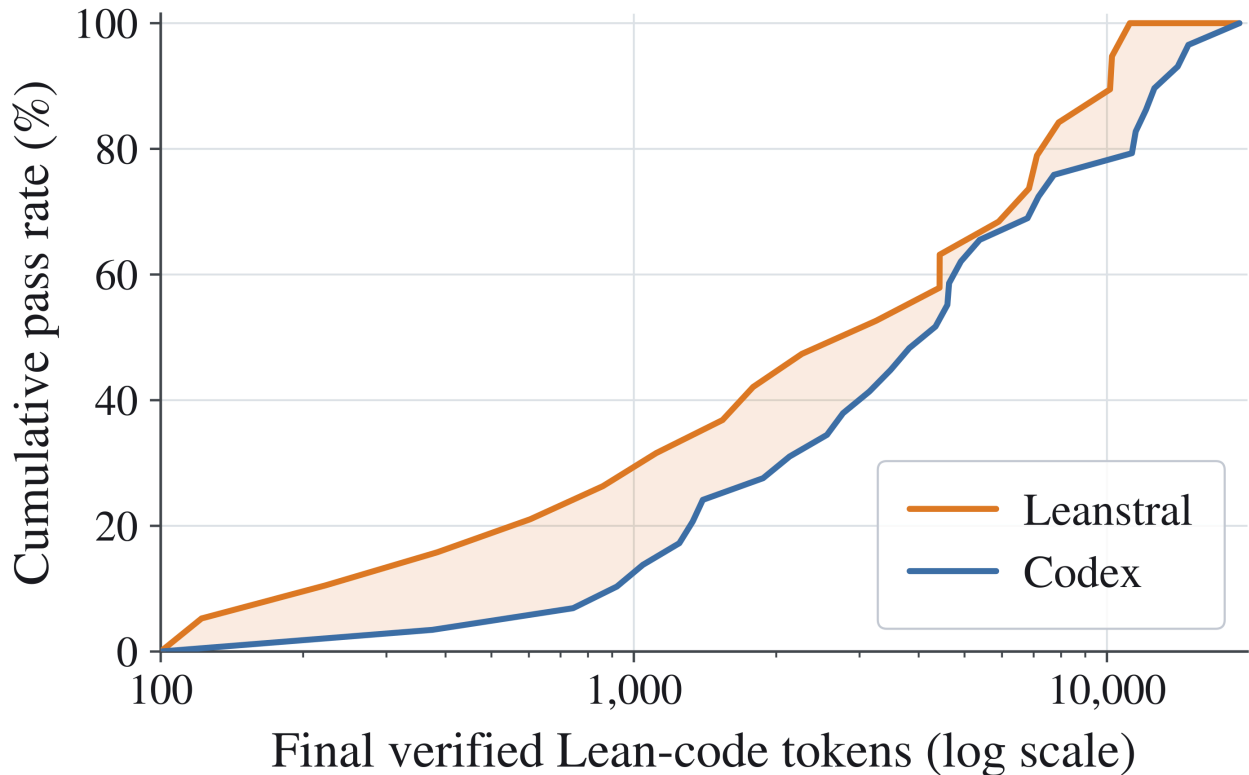


그림 5(d) 검증된 Lean 코드 길이

그림 5(d)를 읽을 때 주의할 점이 하나 있다. 이 패널은 실패가 아니라 성공의 크기를 잴다. 그런데도 격차가 난다. 실패를 많이 겪고 도달한 성공은 흔적을 남기는 반면, 안쪽에서 정돈된 성공은 짧다는 것이다. 산출물의 길이는 곧 이후 검증·유지 보수 비용이므로, 이 격차는 실행 비용 다음에 오는 2차 비용이다.

내부 루프를 크게, 외부 루프를 작게

네 패널을 겹쳐 보면 결론이 하나로 모인다. 두 구성 모두 최종 천장은 같다. 결국 같은 문제를 다 풀 수 있다는 뜻이다. 능력의 상한이 같린 것이 아니라, 같은 상한에 도달하는 예산이 같렸다. 재시도를 안쪽 루프에 두면 소비도 줄고 실패가 바깥으로 번지면서 남기는 오염도 줄고, 검증된 산출물마저 짧아진다. 설계 교훈은 단문으로 정리된다. 내부 루프를 크게, 외부 루프를 작게. 비용의 지형은 이걸로 끝이 아니다. 같은 파이프라인이 문장을 바꿔 받아들일 때도 흔들리지 않는지가 다음으로 물을 질문이다.

DAY

10

바뀌 말해도 무너지지 않는다



같은 수학 내용을 다른 문장으로 쓰면, 모델은 같은 실력을 보일까. 이 질문은 사소해 보이지만 벤치마크의 신뢰를 전부 좌우한다. 숫자가 바뀌지 않는 한 모델이 문제를 푼 것인지 문장을 외운 것이 아니라고 장담하기 어렵기 때문이다. 원문은 AIME 2026 문제를 의미는 보존하되 표현만 바꾼 패러프레이즈(paraphrase) 버전으로 다시 풀어 보는 실험으로 이 물음에 답했다.

베이스 모델은 문장에 흔들린다

결과부터 말하면 베이스 모델은 문장에 흔들렸다. K2-Horizon-375B는 원문제에서 90.0을 기록했지만 패러프레이즈에서 86.7로 3.3포인트 내려왔다. K2-Horizon-7B는 격차가 더 컸다. 원문제 대비 정확도가 열 포인트 통째로 사라졌다. 같은 수학을 묻는 문장이 조금 달라졌을 뿐인데 답이 달라졌다는 것은, 그 모델의 성적 표에 수학 실력과 문장 스타일에 대한 의존이 섞여 들어 있다는 뜻이다.

Magenta 결합은 델타가 0이다

Magenta를 결합하면 그림이 완전히 달라진다. 두 모델 모두 원문제와 패러프레이즈에서 같은 점수를 기록했고 변화량은 정확히 0.0이었다. 이유는 파이프라인의 구조에 있다. Magenta의 최종 검증 대상은 자연어 답변이 아니라 형식 명제다. 명제 판사는 그 형식 명제가 원래 문제와 답을 충실하게 보존하는지를 강제로 심사한다. 표면 언어가 아무리 흔들려도 검증 대상인 명제는 문제의 수학적 내용에서 다시 만들어지므로, 문장 스타일의 변화가 답의 의미까지 끌고 가지 못한다. 답이 문장에 묶이지 않고 명제에 묶인 것이 이 델타 0의 정체다.

이 실험은 오염(contamination) 통제 검사로도 읽힌다. 같은 문제를 다르게 서술하면 기억에 기댈 여지가 줄어든다. 베이스 모델에서 벌어진 하락은 표면 문장에 기대던 비중이 실재했다는 신호고, 델타 0은 파이프라인이 그 표면을 거치지 않는다는 방증이다.

표 10-1은 원문 Table 2를 그대로 옮긴 것이다. AIME 2026 기준이며, 원문제 점수와 패러프레이즈 점수, 그리고 그 차이를 나란히 놓았다.

모델	원문	패러프레이즈	Δ
K2-Horizon-375B	90.0	86.7	-3.3
K2-Horizon-375B + Magenta	100.0	100.0	0.0
K2-Horizon-7B	80.0	70.0	-10.0
K2-Horizon-7B + Magenta	100.0	100.0	0.0

표를 세로로 훑으면 규칙이 보인다. 베이스 행은 모두 음의 델타를 가지고, Magenta 결합 행은 전부 0.0이다. 크기도 모델 크기와 무관하지 않다. 작은 모델일수록 문장 변화에 더 크게 무너지는데, Magenta 결합은 그 격차까지 통째로 지운다. 형식 명제라는 고정된 검증 대상이 모델 크기와 무관하게 표면 노이즈를 흡수한다는 것이다.

여기까지 보면 Magenta의 보증이 상당히 단단해 보인다. 그런데 그 보증이 정말로 명제 판사에서 나오는지는, 판사를 빼고 돌려 봐야 알 수 있다. 다음 장은 바로 그 절제 실험을 연다.

DAY

11

판사를 끼웠을 때와 겼을 때



Magenta의 보증이 실제로 명제 판사에서 나오는지 확인하려면, 판사를 빼고 같은 일을 다시 시켜 봐야 한다. 원문은 Goedel 형식화기를 엮은 구성에서 명제 판사를 끼운 경우와 끈 경우를 AIME 2026 문제로 나란히 돌렸다. 배치 크기는 32다. 이 절제 실험이 남긴 숫자는 이 책 전체를 관통하는 교훈의 증거가 된다. 검증을 통과했다는 것과 정답이라는 것은 다르다.

세 가지 지표를 먼저 정의하자

비교에 쓰인 지표는 셋이다. 검증 통과(Ver)는 증명이 Lean 검증기를 통과했는지를 센다. 검증·일치(VerCor)는 검증 통과에 더해 그 명제가 원래 문제와 실제로 일치하는지까지 요구한다. 거짓 증명서 비율(false certification rate, FCR)은 검증을 통과한 것 가운데 원문제와 일치하지 않는 비중으로, 1에서 VerCor를 Ver로 나눈 값을 뺀 것이다. 요컨대 FCR이 0이라는 말은, 검증기가 통과시킨 증명서에 가짜가 하나도 없다는 뜻이고, FCR이 크다는 말은 검증기가 통과시킨 것의 상당 부분이 엉뚱한 것의 증명이라는 뜻이다.

표 11-1은 원문 Table 3을 옮긴 것이다. 명제 판사의 유무에 따라 세 지표가 어떻게 달라지는지 보여 준다.

설정	Ver(검증 통과)	VerCor(검증·일치)	FCR(거짓 증명서 비율)
명제 판사 있음	63.3	63.3	0.0
명제 판사 없음	36.7	20.0	45.5

판사가 있을 때 Ver과 VerCor가 같은 값을 가진다는 사실이 핵심이다. 검증을 통과한 증명서가 전부 원문제와 일치했다는 뜻이다. 판사를 끄면 두 지표가 함께 무너진다. Ver 하락을 시도 감소로 돌려서는 안 된다. 심사를 없애면 첫 명제가 무조건 증명 단계로 넘어가므로 시도 자체는 줄지 않는다. 줄어든 것은 그 시도 가운데 검증을 통과하는 증명의 비율이다. 더 심각한 것은 남은 검증 통과분의 거의 절반이 가짜 증명서라는 사실이다. FCR 45.5퍼센트다.

표 11-1의 두 하락과 한 상승은 사실 하나의 효과가 세 갈래로 기록된 것이다. 명제 심사는 첫째로 형식화의 품질 필터로 일한다. 부실한 명제를 증명 단계로 보내지 않으므로, Ver의 하락 폭이 심사가 걸러 낸 것의 양을 그대로 보여 준다. 둘째로 거짓 증명서 차단으로 일한다. 통과시킨 명제가 원문제와 일치하는지 따져 VerCor를 Ver에 맞춰 붙인다. 판사를 끄는 순간 이 두 역할이 동시에 사라지고, Ver과 VerCor는 갈라져 벌어지며, 그 간격이 FCR이라는 청구서로 정산된다. 있음 행에서 두 지표가 겹쳐 있는 것과 없음 행에서 두 지표가 벌어진 것의 차이가 곧 명제 심사의 가격표다.

거짓 증명 다섯 건의 해부

원문은 판사를 끈 구성에서 반환된 거짓 증명서를 건별로 감식했다. 다섯 건 가운데 세 건은 문제를 악화시키거나 답을 명제 안에 몰래 내장하는 대리 목표를 증명하고 있었다. 한 건은 문제의 조건 일부를 누락한 채 증명을 완성했고, 한 건은 기하 구조를 오역해 다른 정리를 증명하고 있었다. 모순된 문맥을 통과한 사례는 0건이었다. 요컨대 가짜들은 우당탕대는 무리수가 아니라, 그럴듯하게 생긴 다른 문제의 정당한 증명이었다.

이 감식이 왜 무서운지는 증명기의 성격을 떠올리면 답이다. 증명기는 명제가 쓴 대로만 참이라 본다. 검증기는 그 명제가 원래 문제를 말하는지까지 책임지지 않는다. 명제가 슬쩍 약해진 문제를 담고 있으면, 증명기는 그 약화된 명제를 완벽하게 증명해 내고 검증기는 박수를 보낸다. 대리 목표를 노린 다섯 건 중 세 건이 바로 이 경로로 들어왔다. 형식적 완결성과 의미적 충실함 사이의 이 간극을 메우는 장치가 명제 판사인 셈이다.

검증 통과는 정답 인정이 아니다

절제 실험의 교훈을 한 문장으로 압축하면 이렇다. 검증 통과는 정답 인정이 아니라, 쓰여진 명제에 대한 국소적 합격 판정일 뿐이다. 판사를 끄는 때 반환된 증명서의 절반 가까이가 거짓이었다는 사실은, 검증기의 침묵을 성실함으로 오독하면 안 된다는 경고다. 기계가 검사하는 증명이라는 이 책의 주인공은, 명제가 올바르게 쓰였을 때에만 진짜 힘을 발휘한다. 명제의 충실함을 지키는 별도의 심사가 없다면 검증은 정확하게, 그러나 엉뚱한 대상을 보증한다.

지금까지 판사가 보증의 경계를 어디까지 미는지 봤다. 파이프라인 안에는 판사가 하나 더 있다. 실패를 수학과 구현으로 갈라 라우팅하는 오류 귀인 판사다. 이 판사가 만든 피드백 루프가 단순히 다시 뽑는 방식과 어떻게 다른 성적을 내는지가 다음 장의 주제다.

DAY

12

고치기와 다시 뽑기



검증이 실패했을 때 시스템이 취할 수 있는 가장 값싼 행동은 같은 추론기를 다시 굴리는 일이다. 실패한 궤적을 버리고 새로 뽑는 이 방식을 재샘플링(resampling)이라고 부르겠다. 직관은 착각을 낳는다. 다시 뽑으면 언젠가는 맞는 답이 나올 것 같은 느낌이 든다. 그러나 원문의 절제 실험은 반대를 증명한다. 어려운 문제로 갈수록 다시 뽑기는 천장에 부딪히고, 오류 궤적을 물려받아 그 지점만 고치는 자기수정(self-correction)이 앞선다.

포화되는 확률

성공확률이 p 인 독립 시도를 n 회 굴리면 적어도 한 번은 성공할 확률은 다음과 같다.

$$1 - (1 - p)^n \geq 1 - e^{-np}$$

부등식이 말해 주는 것은 단순하다. 시도 횟수를 늘리면 성공확률이 1에 다가가지만, 그 접근은 지수적으로 둔화되어 포화된다. 처음 몇 번의 재시도가 남긴 이득이 가장 크고, 그 뒤로는 토큰만 태운다. 더 근본적인 균열은 독립이라는 가정에 있다. 같은 추론기에 같은 문제를 다시 주면, 모델은 같은 함정에 같은 방식으로 빠지기 쉽다. 지문의 특정 표현이 특정 잘못된 해석을 부르는 경향은 시도마다 그대로 재생된다. 확률론이 약속하는 독립성이 실제 추론기에서는 성립하지 않으므로, 포화는 이론치보다 더 빨리 온다.

AIME 2026의 한 문제 차이

실측이 이 그림을 그대로 좇는다. AIME 2026 서른 문제에서 재샘플링은 29문제를 풀었다. 29/30, 즉 96.7퍼센트다. 자기수정은 30/30으로 전부 풀었다. 격차는 한 문제다. 그러나 그 한 문제가 이 방식의 본질을 드러낸다. 원문의 설명은 이렇다. 그 문제에서는 초기 답이 틀린 상태로 명제가 세워졌고, 답이 거짓인 명제에는 참인 증명이 존재하지 않으므로 증명 샘플을 더 뽑아도 회복할 수 없었다. 재샘플링이 답 단계의 오류를 고치려면 우연히 옳은 답이 다시 나와야 하고, 확률은 앞 절에서 본 대로 포화되어 있다.

두 방식의 차이는 정보의 취급에서 시작한다. 재샘플링은 실패를 정보로 쓰지 않는다. 이전 시도가 어디서 무너졌는지 기록을 남기지 않은 채 궤적을 통째로 폐기하고 처음부터 다시 쓴다. 자기수정은 반대로 실패를 자산으로 편입한다. 검증기가 찾아 낸 오류 지점과 그 성격이 다음 시도의 출발선이 된다. 같은 예산이 돌아도 한쪽은 매번 영에서 출발하고 다른 쪽은 누적된 지식 위에서 출발한다.

IMO 2026에서 벌어지는 격차

문제가 어려워지면 격차는 한 문제에서 여섯 문제로 벌어진다. IMO 2026 여섯 문제에서 재샘플링은 문제당 8~16회의 독립 생성을 허용받았다. 그 결과는 1/6, 즉 16.7퍼센트다. 한 문제를 풀었다. 자기수정은 여섯 문제 전부를 풀었다. 같은 추론기, 같은 검증기, 같은 예산에서 결과가 이만큼 갈린다. 어려운 문제일수록 초기 궤적이 함정에 빠질 확률이 커지고, 다시 뽑기는 그 함정을 시도마다 되풀이한다. 문제당 8~16회라는 배려심 어린 예산조차 이 되풀이를 막지 못했다. 자기수정은 길이 다르다. 검증기가 남긴 오류 기록을 다음 시도에 물려주고, 궤적 전체를 다시 쓰는 일 없이 그 지점만 고친다. 실패의 위치를 아는 시도와 위치를 모르는 시도의 차이가 곧 성패의 차이이다.

고칠 거리와 다시 뽑을 거리

이 장의 교훈을 한 문장으로 압축하면 이렇다. 검증 피드백이 존재하는 한, 버리고 다시 시작하는 비용은 필요한 부분만 고치는 비용을 넘지 못한다. 물론 자기수정이 만능은 아니다. 오류가 수학 자체에 있으면 처음부터 다시 추론해야 하고, 그 판단을 내주는 것이 앞 장에서 본 오류 귀인 판사다. 라우팅이 있기에 고치기가 성립한다. 그렇다면 실패는 실제로 어디에 물려 있을까. 다음 장은 AIME 2026에서 기록된 실패 전체를 세 겹의 표로 해부하고, 그 지형 위에서 사람과 기계가 같은 곳에서 넘어지는 장면을 조명한다.

DAY

13

오류의 해부 — 실패 지형과 형식화 갭



완성의 표만 놓고 보면 파이프라인이 한 번에 맞아 떨어진 것처럼 보인다. 그러나 100퍼센트라는 최종 결과 뒤에는 셀 수 없이 많은 실패가 흘렀다. 원문은 AIME 2026 서론 문제에서 일어난 실패를 세 겹으로 기록했다. 명제가 만들어지고 심사 받는 단계, 증명이 시도되는 단계, 그리고 그 밑바닥에서 수학 오류가 태어나는 근원이다. 이 장은 부록의 세 표를 압축해 그 지형을 따라 걷는다. 실패의 분포를 알면 시스템이 왜 이렇게 설계되었는지도 함께 보인다.

명제 단계 — 컴파일은 충실성의 약한 대리

형식화기가 뽑은 명제의 66.55퍼센트는 Lean 엘라보레이트(elaborate)를 통과하고, 33.08퍼센트는 컴파일 오류로 심사 이전에 걸러진다. 관건은 통과한 명제의 운명이다. 형식화기가 뽑은 전체 명제의 44.64퍼센트가 심사에서 의미 거절되고 21.91퍼센트가 수용된다. 컴파일을 넘겨 심사까지 간 명제만 놓고 보면 거절이 67.08퍼센트, 수용이 32.92퍼센트다. 앞의 두 값은 전체 생성을 기준으로 한 것이고 뒤의 두 값은 심사 대상만을 기준으로 한 것이므로 저울질할 때 기준을 섞지 않아야 한다. 어느 기준을 따라도 심사가 컴파일보다 더 많이 걸러낸다. 이 비대칭이 이 절의 논점이다. 컴파일은 Lean 문법에 맞게 적혔지만 확인한다. 원문제를 충실하게 옮겼는지는 컴파일의 사정거리 밖이다. 그래서 컴파일 통과는 충실성의 약한 대리(weak proxy)일 뿐이고, 그 빈자리를 명제 판사가 메운다. 아래 표는 이 단계의 오류를 생성 기준과 조건부 기준으로 나눈 것이다.

표 13-1 명제 생성·심사 오류(AIME 2026 전체 30문제, 원문 Table 6 압축)

결과·범주	생성 비율(%)	조건부 비율(%)
Lean 명제 완성	66.55	—
Lean 컴파일 오류	33.08	—
타입·강제·프로젝션 미스매치	17.47	52.23
타입클래스·인스턴스 합성	7.85	23.46
파서·표기법 구문	4.78	14.28
알 수 없는 이름·API 오용	2.73	8.17
의미 거절(심사 판사)	44.64	67.08
의미 수용(심사 판사)	21.91	32.92

컴파일 오류의 내역도 뚜렷한 경향을 보인다. 컴파일이 실패한 경우를 조건부 기준으로 재계산하면 절반을 넘는 52.23퍼센트가 타입·강제(coercion)·프로젝션 미스매치다. 뒤를 타입클래스·인스턴스 합성 실패 23.46퍼센트, 파서·표기법 14.28퍼센트, 알 수 없는 이름 8.17퍼센트가 잇는다. 명제 단계의 실패는 대부분 수학이 틀려서가 아니라 수학을 Lean의 언어로 표현하는 기계적 접합이 어긋나서 일어난다.

증명 단계 — 도구마다 다른 실패 프로파일

명제를 통과하면 증명기가 무대에 오른다. 원문은 세 가지 증명 구성, 즉 Leanstrat을 내부 API로 쓴 구성과 외부 API로 쓴 구성, 그리고 Codex를 외부로 쓴 구성의 오류 분포를 따로 기록했다. 세 구성 모두에서 최대 실패 유형은 전술 실패와 열린 목표다. 그러나 값은 42.70, 63.78, 30.28로 제각각이고, 그 아래로 깔린 실패의 조합도 도구마다 다르다. Leanstrat 외부 구성에서는 금지 토큰·정책 위반이 26.77퍼센트나 차지한다. SafeVerify가 증명에 증명 회피 토큰이나 새 공리를 감지하면 정책 발동으로 기각하는 것이다. Codex 외부 구성은 양상이 완전히 달라 타입·강제 미스매치가 45.92퍼센트로 최대다. 같은 과제를 주어도 증명 도구의 습성이 실패의 지형을 다시 그린다. 아래 표는 세 구성의 오류 분포를 나란히 놓은 것이다.

표 13-2 증명 오류 분포(AIME 2026 전체 30문제, 원문 Table 7 압축)

오류 범주	Leanstrat 내부	Leanstrat 외부	Codex 외부
전술 실패·열린 목표	42.70	63.78	30.28
금지 토큰·정책	—	26.77	—

오류 범주	Leanstral 내부	Leanstral 외부	Codex 외부
타입·강제 미스매치	6.70	–	45.92
알 수 없는 이름·API 오용	15.19	–	15.31
도구·프로토콜·런타임	13.68	–	2.16
파서·구문	8.18	0.79	0.67
자원 한도·시간 초과	5.72	8.66	2.66
타입클래스·인스턴스 합성	2.60	–	1.83

표를 다시 읽어 보면 라우터와 수리 설계가 왜 구현 오류와 수학 오류를 구분해야 하는지도 답이 나온다. 파서·구문이나 자원 한도 같은 항목은 증명만 다시 쓰면 해결되는 구현 실패다. 전술 실패·열린 목표는 증명 전략이 막혔다는 신호이고, 심하면 답부터 다시 세워야 한다. 실패 범주가 곧 라우팅의 근거다.

수학 오류의 근원 — 14건의 귀인

마지막 겹은 가장 얇지만 가장 깊다. 오류 판사가 수학 오류로 귀인한 사례는 AIME 2026에서 열네 건이다. 그 근원 분포는 다음과 같다.

표 13-3 수학 오류 근원(K2-Horizon-7B, 귀인 14건, 원문 Table 8)

근원	비율(%)
입력 해석·전사	28.6
뒷받침 없는 논리·증명 단계	28.6
산수·대수·크기 계산	21.4
반복 수리로 계약 보존 실패 후 에스컬레이션	14.3
반례 구성·잘못된 구성	7.1

입력 해석과 뒷받침 없는 논증 단계가 공동 최대라는 사실이 주목할 만하다. 둘을 합치면 절반을 넘긴다. 기계는 계산보다 읽기에서 더 자주 넘어진다.

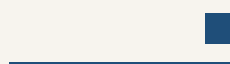
형식화 갭 — 사람과 기계가 같은 곳에서

이 지형을 관통하는 공통 조건이 있다. 형식화 갭(autoformalisation gap)이다. 원문이 예로 든 오류를 나열해 보면 17017을 107017로 바꿔 적는 치환, 천을 만으로 읽는 전사(1000을 10000으로), 3의 아홉 제곱을 39로 붙여 쓰는 표기, 반복 횟수를 6회에서 7회로 해석하는 실수다. 숫자를 옮기는 짧은 구간에서 의미가 뒤틀린다. 이 실수 목록을 개발자의 회의록에서 본 듯한 느낌을 받았다면 옳게 본 것이다. 요구사항을 읽고 테스트를 쓰는 사람도 같은 자리에서 같은 실수를 한다. 형식화 갭은 기계의 약점이 아니라 자연어를 기호로 옮기는 모든 행위의 약점이다. 파이프라인이 명제 판사와 오류 판사를 둘이나 둔 이유가 여기에 있다. 다음 장은 이 통찰을 우리 현장의 관행에 옮겨 본다.

DAY

14

한국 현장에서 읽는 Magenta



이 장은 필자의 각색이며 원문 저자의 주장이 아니다. 원문은 수학 추론과 Lean 검증의 파이프라인을 다룰 뿐, 한국의 개발 문화나 실무 품질 관행을 말하지 않는다. 필자가 원문의 개념을 우리 현장에 옮겨 본 것이고, 원문에서 확인한 수치만 그대로 재인용한다.

증명서의 이식 — 답 말고 근거를 봉인하라

Magenta가 남기는 최종 산출물은 답이 아니라 증명서다. 답과 함께 그 답을 Lean 명제로 봉인하고 기계가 검사한 증명을 붙인다. 이 구조를 금융·공공·제조의 품질 파이프라인에 이식해 보자. 심사 통과 알림 한 줄로 끝내는 대신, 산출물에 계산 근거와 적용된 규칙 인용을 붙여 누구나 다시 검산할 수 있는 형태로 남기는 관행이다. 검증 가능한 형태로 봉인된 결과는 인수인계와 감사에서 힘을 쓴다. 답만 남긴 결과는 만든 사람이 없으면 근거도 함께 사라진다.

판사의 이중화 — 평가 루브릭에 거절 사유를 심어라

LLM 판사가 요즘 사내 평가 파이프라인의 표준 장비가 되었다. 그러나 원문은 판사도 실패한다는 사실을 숫자로 남겼다. 명제 심사를 곧 설정에서 반환된 증명서의 거짓 비율, 즉 FCR(false certification rate)은 45.5퍼센트였다. 심사를 켜자 0.0퍼센트로 떨어진다. 절반 가까운 거짓 증명서가 심사 판사 한 명으로 걸러진 셈이다. 이 교훈을 사내 평가 루브릭에 심는 방법이 있다. 통과·실패의 이분법 판정에 그치지 않고, 실패 판정에 거절 사유 분류를 의무화하는 것이다. 원문의 오류 귀인 판사가 실패를 수학과 구현으로 갈라 갈 길을 정하듯, 평가 담당자가 거절 사유를 유형별로 기록해야 개선 라우팅이 가능하다. 판사를 도입하는 팀은 판사의 판사까지 설계해야 한다.

재시도 예산의 비대칭 — 어디에 다시 굴릴 것인가

운영에서 늘 부딪히는 질문이 있다. 실패했을 때 예산을 어디에 쓸 것인가. 원문의 비용 분석이 답의 형태를 준다. 외부에 재시도를 두면 전체 커버리지 기준 리즈너 토큰이 약 9×10^5 에 달하고, 검증 피드백을 물려받는 경로는 약 6×10^4 에서 끝난다. 내부 수리는 검증된 증명 길이도 약 2×10^4 을 1.1×10^4 가량으로 줄인다. 같은 성공을 사는 두 가격표다. 외부 재시도는 문제를 처음부터 다시 풀고, 내부 수리는 실패 지점부터 이어서 간다. 재시도 예산을 설계할 때 이 비대칭을 근거로 삼을 만하다. 피드백을 전달할 수 있는 실패라면 고치는 쪽이 싸고, 피드백이 무의미한 실패라면 다시 뽑는 쪽이 낫다.

형식화 갭은 사람도 겪는다

앞 장에서 본 형식화 갭 사례를 떠올려 보자. 17017을 107017로 바꿔 적는 치환 같은 오류다. 이 실수는 모델의 전유물이 아니다. 요구사항 문서와 테스트 코드 사이에서 숫자를 옮기는 사람의 작업이 정확히 같은 구간이다. 한도 금액, 기준일, 임계값이 문서에서 코드로 전사되는 자리마다 같은 종류의 뒤틀림이 가능하다. 그래서 형식화 갭은 모델 정확도의 문제로만 다루지 말고 전사 검증의 문제로 다루어야 한다. 원문이 명제 판사로 명제의 충실성을 따로 심사하듯, 요구사항과 코드 사이에도 숫자 일치를 검사하는 절차가 독립으로 존재할 필요가 있다.

작은 모델과 루프, 큰 모델과 단발

조달 담당자에게 가장 실질적인 결과는 아마 이것일 것이다. 오픈 웨이트 7B 모델 K2-Horizon-7B에 Magenta를 붙이자 AIME 2025, AIME 2026, HMMT 2026의 93문제 전부에서 100퍼센트를 기록했고, IMO 2026 여섯 문제도 전부 풀었다. 단발 능력이 최상위인 거대 모델을 사는 대신, 작은 모델에 검증 루프를 씌우는 선택지가 실재한다. 물론 이 결과가 즉시 모든 도메인에 일반화되지는 않는다. 수학 문제처럼 검증기가 명확한 영역이기에 가능한 승리다. 그러나 검증 가능성을 확보할 수 있는 영역에서는 조달의 판단 기준이 모델 크기에서 루프 설계로 옮겨 간다. 작은 모델과 단단한 루프가 큰 모델의 단발을 이긴다는 사례로 기억할 만하다.

원문의 개념을 현장에 두다 보면 공통의 바닥이 보인다. 검증을 신뢰하고, 판사를 이중화하고, 실패의 위치에 예산을 쓴다는 것. 마지막 장은 이 책 전체를 한 판에 정리하고 남은 과제를 짚는다.

DAY

15

맺음말



이 책은 한 논문을 따라 걸었다. 자연어로 쓰인 수학 문제가 답이 되고, 명제가 되고, 기계가 검사한 증명이 되는 여정이다. 마지막 장에서는 그 여정을 결론의 축부터 다시 세우고, 파이프라인 전체를 한 판에 정리한 뒤, 아직 남은 갭과 한계를 짚는다.

세 개의 축

원문의 결론은 세 개의 축으로 요약된다. 첫째 축은 루프 폐쇄다. 비형식 추론과 형식 검증은 서로를 대체하는 경쟁 관계가 아니라 상호 보완이다. 추론기는 문제를 읽고 답을 세우고, Lean은 그 결과를 이산적으로 검증한다. 검증의 신호가 다시 추론에 흘러들 때 두 능력이 하나의 시스템이 된다. 학습도 튜닝도 없이 프롬프트로만 이 루프를 조립했다는 점에서 training-free라는 수식어가 붙는다. 둘째 축은 명제 심사의 필수성이다. 심사를 끄면 거짓 증명서가 돌아왔고, 그 다섯 건 중 세 건은 목표를 약화하거나 답을 내장한 대리 목표였다. 나머지는 문제 조건 누락과 기하 오역이었고, 모순된 문맥을 통과한 증명은 없었다. 증명이 옳아도 다른 것을 증명했으면 증명서가 아니다. 셋째 축은 피드백 수정이 독립 재샘플링을 이긴다는 사실이다. 어려운 문제일수록 격차는 벌어졌다. AIME 2026에서는 재샘플링도 30문제 중 29개를 검증해 격차가 좁아 보였다. 그러나 IMO 2026에서는 문제당 여덟 번에서 열여섯 번의 독립 생성을 쏟아부어도 여섯 중 하나에 그쳤고, 자기수정은 여섯 전부를 검증했다. 같은 모델을 같은 방식으로 다시 굴리는 일과, 실패한 지점의 궤적을 물려받아 그 지점만 고치는 일은 다른 활동이라는 뜻이다.

한 판에 정리

구성요소로 돌아가 전체를 짚는다. 리즈너가 최대 32회의 재추론 라운드에서 문제를 읽고 답을 세운다. 형식화기가 명제 후보를 최대 512개 뽑고, 명제 판사가 원문제와 답을 보존하는 후보만 통과시킨다. 증명기가 최대 4096회의 시도 안에서 증명을 완성하고, 검증기가 명제와 증명을 Lean 위에서 검사한다. 실패가 나면 오류 귀인 판사가 실패를 수학과 구현으로 갈라 전자는 재추론으로, 후자는 증명 수리로 보낸다. 바닥에는 Lean v4.29.1과 Mathlib, 그리고 증명 회피 토큰과 신규 공리를 막는 SafeVerify가 깔려 있다. 세 겹의 루프가 이 일곱 구성요소를 실행 돈다. 답의 루프, 명제의 루프, 증명의 루프다. 결과는 세 벤치마크 93문제 전부와 IMO 2026 여섯 문제의 완전 검증이었다.

남은 갭과 다음 독자

성실함을 위해 한계도 분명히 하자. 검증 범위는 경시대회 규모의 93문제와 IMO 여섯 문제다. 판사 역할이 DeepSeek 한 종류에 의존하므로 심사 품질이 판사 모델의 품질을 따라간다. training-free는 학습 비용이 없다는 뜻이지 검증 비용이 없다는 뜻이 아니며, 방대한 재추론과 증명 시도 예산이 그 대신 든다. IMO 2026에서 7B 모델은 문제마다 평균 5.7번의 자기수정을 거쳤고, 가장 고된 문제는 스무 번의 루프를 돌았다. 오염 의심에 대한 답도 조건부다. 바꿔 쓴 문제에서 성적이 흔들리지 않았다는 증거는 필요조건이지 충분조건이 아니다. 이 시험이 통과됐다고 오염이 완전히 배제된 것은 아니다. 숫자 자체를 바꾼 시험까지 통과해야 확증이 된다. 그리고 원문이 스스로 가리키는 다음 관문이 있다. 명제가 원문제를 충실하게 옮기는 일, 즉 형식화 갭을 닫는 일이다. 이 책에서 본 오류 지형은 그 갭이 아직 남아 있음을 보여 주었다. 루프가 닫혀도 루프에 들어가는 명제가 문제와 다르면 완벽한 증명도 빈 증명서다.

다음 독자에게 권하고 싶은 읽기가 있다. 논문을 하나의 결과가 아니라 하나의 설계로 읽으라. 검증 가능한 목표를 세우고, 실패를 귀인하고, 귀인에 따라 고치거나 다시 시작하는 설계다. 이 구조는 수학 밖으로 옮길 수 있는 부분이 충분하다. 모델이 무엇을 말하든 검증기가 무엇을 통과시켰는지 묻는 습관, 그것이 이 논문이 남긴 가장 이식성 높은 유산이다.